post_experiment_analysis

March 29, 2026

1 Post-Experiment Analysis

Standard notebook for vulnerability classification runs with emphasis on CWE generalization, conservatism vs over-prediction, weak buckets, threshold sensitivity, and repeatable comparisons.

Interpretation rules encoded here: - Accuracy can mislead because vulnerability data is usually imbalanced. - `unseen_test` is the key split for CWE generalization. - High unseen precision with lower unseen recall usually means the model is conservative on unseen CWE. - Per-CWE metrics matter because aggregate metrics can hide failures in rare classes. - Threshold analysis matters because recall/F1 may improve without retraining.

[1]:
```
import os
os.chdir("/Users/noamc/repos/graph-vul")
```

[2]:
```
from pathlib import Path

# Minimal config edits for each future run.
RUN_NAME = "func_finetune_model_ckpt"
MODEL_NAME = "microsoft/unixcoder-base + CWE embedding fusion (fine-tuned)"
DATASET_NAME = "data/func_subset.csv with data/func_splits/assignments.csv"
UNSEEN_DEFINITION = "Held-out CWE categories (CWE-200, CWE-416) excluded from␣
 ↪train/val/test and evaluated in unseen splits."
CAVEATS = [
    "Per-row prediction files are required for full analysis.",
    "If only hard labels exist, score and threshold sections will be skipped.",
]

PREDICTION_PATHS = {
    "val": "data/analysis/func_finetune_predictions/val_predictions.csv",
    "test": "data/analysis/func_finetune_predictions/test_predictions.csv",
    "unseen_test": "data/analysis/func_finetune_predictions/
 ↪unseen_test_predictions.csv",
    # Optional if your run produces these:
    "unseen_val": "data/analysis/func_finetune_predictions/
 ↪unseen_val_predictions.csv",
    "train": None,
}
```

```
OPTIONAL_METRICS_PATHS = {
    "metrics_json": None,
}

THRESHOLD = 0.50
SCORE_KIND_OVERRIDE = "auto"  # "auto", "prob", or "logit"
THRESHOLD_GRID = [round(x, 2) for x in __import__("numpy").linspace(0.05, 0.95,␣
 ↪19)]
N_BOOTSTRAP = 1000

OUTPUT_DIR = Path("outputs") / "post_experiment_analysis" / RUN_NAME
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

print(f"Run: {RUN_NAME}")
print(f"Output dir: {OUTPUT_DIR.resolve()}")
```

```
Run: func_finetune_model_ckpt
Output dir: /Users/noamc/repos/graph-
vul/outputs/post_experiment_analysis/func_finetune_model_ckpt
```

1.1 Overview

Model: {{ MODEL_NAME }}
Dataset / subset: {{ DATASET_NAME }}
What counts as unseen: {{ UNSEEN_DEFINITION }}

Update the config cell above with the actual values for this run. This notebook assumes per-row prediction files by split and will skip sections that are not supported by the available columns.

[3]:
```
import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from IPython.display import Markdown, display
from sklearn.metrics import confusion_matrix as sk_confusion_matrix

from graph_vul.analysis import post_experiment_analysis as pea

pd.set_option("display.max_columns", 100)
pd.set_option("display.max_rows", 200)
plt.style.use("default")
np.random.seed(pea.RANDOM_SEED)
```

1.2 Load Predictions

This section validates the input files before any metric interpretation. It checks shapes, columns, class balance, unique CWE/project counts, duplicate IDs when available, null predictions, and malformed probability values.

If the current run output schema is missing fields such as `cwe_id`, `project`, `is_unseen_cwe`, scores, or metadata lengths, the notebook will emit a TODO-style warning and skip only the affected analyses.

[4]:
```
frames, load_metadata = pea.load_prediction_splits(
    PREDICTION_PATHS,
    threshold=THRESHOLD,
    score_kind_override=SCORE_KIND_OVERRIDE,
)
frames = pea.derive_unseen_flags(frames)

if not frames:
    raise FileNotFoundError("No prediction files were loaded. Update␣
 ↪PREDICTION_PATHS in the config cell.")

display(Markdown("### Run Metadata"))
display(pd.DataFrame({
    "field": ["run_name", "model_name", "dataset_name", "unseen_definition",␣
 ↪"threshold"],
    "value": [RUN_NAME, MODEL_NAME, DATASET_NAME, UNSEEN_DEFINITION, THRESHOLD],
}))
if CAVEATS:
    display(Markdown("### Caveats"))
    for caveat in CAVEATS:
        display(Markdown(f"- {caveat}"))

split_summaries = []
for split_name, frame in frames.items():
    split_summary = pea.summarize_split_frame(frame)
    split_summary.name = split_name
    split_summaries.append(split_summary)
display(Markdown("### Split Summary"))
display(pd.DataFrame(split_summaries))

for split_name, frame in frames.items():
    display(Markdown(f"### `{split_name}` column summary"))
    display(pd.DataFrame({"column": frame.columns, "dtype": [str(frame[c].
 ↪dtype) for c in frame.columns]}))
    display(Markdown(f"**Label balance for `{split_name}`**"))
    label_balance = frame["y_true"].value_counts(dropna=False).
 ↪rename_axis("label").reset_index(name="count")
```

```
    label_balance["share"] = label_balance["count"] / label_balance["count"].
 ↪sum()
    display(label_balance)
    display(Markdown(f"**Validation checks for `{split_name}`**"))
    display(pea.validation_checks(frame))

design_checks = pea.confirm_unseen_split_design(frames)
if not design_checks.empty:
    display(Markdown("### Seen vs unseen split design checks"))
    display(design_checks)
else:
    print("TODO: Could not confirm seen/unseen split design because␣
 ↪`is_unseen_cwe` or `cwe_id` is missing.")
```

1.2.1 Run Metadata

field	value
run_name	func_finetune_model_ckpt
model_name	microsoft/unixcoder-base + CWE embedding fusio…
dataset_name	data/func_subset.csv with data/func_splits/ass…
unseen_definition	Held-out CWE categories (CWE-200, CWE-416) exc…
threshold	0.5

1.2.2 Caveats

- Per-row prediction files are required for full analysis.
- If only hard labels exist, score and threshold sections will be skipped.

1.2.3 Split Summary

	rows	columns	positive_rate	unique_cwes	unique_projects	score_available
val	8465	20	0.046545	36	1	True
test	4727	20	0.023482	15	8	True
unseen_test	14430	20	0.060291	51	31	True
unseen_val	15487	20	0.063602	41	19	True

1.2.4 `val` column summary

column	dtype
sample_id	int64
project	object
func_before	object

3	cve_id	object
4	cwe_id	object
5	vul	int64
6	split	object
7	is_unseen_cwe	Int64
8	row_id	int64
9	label	int64
10	char_length	int64
11	line_count	int64
12	y_true	Int64
13	logit	float64
14	prob	float64
15	y_pred	Int64
16	score	float64
17	token_length	int64
18	score_raw	float64
19	code_preview	object

Label balance for `val`

	label	count	share
0	0	8071	0.953455
1	1	394	0.046545

Validation checks for `val`

	check	value
0	missing_y_true	0
1	missing_y_pred	0
2	missing_score	0
3	malformed_probability_values	0
4	duplicate_row_id	0
5	missing_cwe_id	0
6	missing_project	0

1.2.5 `test` column summary

	column	dtype
0	sample_id	int64
1	project	object
2	func_before	object
3	cve_id	object
4	cwe_id	object
5	vul	int64
6	split	object
7	is_unseen_cwe	Int64
8	row_id	int64
9	label	int64
10	char_length	int64
11	line_count	int64

12	y_true	Int64
13	logit	float64
14	prob	float64
15	y_pred	Int64
16	score	float64
17	token_length	int64
18	score_raw	float64
19	code_preview	object

Label balance for `test`

	label	count	share
0	0	4616	0.976518
1	1	111	0.023482

Validation checks for `test`

	check	value
0	missing_y_true	0
1	missing_y_pred	0
2	missing_score	0
3	malformed_probability_values	0
4	duplicate_row_id	0
5	missing_cwe_id	0
6	missing_project	0

1.2.6 `unseen_test` column summary

	column	dtype
0	sample_id	int64
1	project	object
2	func_before	object
3	cve_id	object
4	cwe_id	object
5	vul	int64
6	split	object
7	is_unseen_cwe	Int64
8	row_id	int64
9	label	int64
10	char_length	int64
11	line_count	int64
12	y_true	Int64
13	logit	float64
14	prob	float64
15	y_pred	Int64
16	score	float64
17	token_length	int64
18	score_raw	float64
19	code_preview	object

Label balance for `unseen_test`

	label	count	share
0	0	13560	0.939709
1	1	870	0.060291

Validation checks for `unseen_test`

	check	value
0	missing_y_true	0
1	missing_y_pred	0
2	missing_score	0
3	malformed_probability_values	0
4	duplicate_row_id	0
5	missing_cwe_id	0
6	missing_project	0

1.2.7 `unseen_val` column summary

	column	dtype
0	sample_id	int64
1	project	object
2	func_before	object
3	cve_id	object
4	cwe_id	object
5	vul	int64
6	split	object
7	is_unseen_cwe	Int64
8	row_id	int64
9	label	int64
10	char_length	int64
11	line_count	int64
12	y_true	Int64
13	logit	float64
14	prob	float64
15	y_pred	Int64
16	score	float64
17	token_length	int64
18	score_raw	float64
19	code_preview	object

Label balance for `unseen_val`

	label	count	share
0	0	14502	0.936398
1	1	985	0.063602

Validation checks for `unseen_val`

	check	value
0	missing_y_true	0

```
1                missing_y_pred      0
2                 missing_score      0
3  malformed_probability_values      0
4              duplicate_row_id      0
5                missing_cwe_id      0
6               missing_project      0
```

1.2.8 Seen vs unseen split design checks

```
                        check value
0     unseen_test_all_heldout  True
1  test_contains_only_seen_cwe  True
2            test_unseen_share   0.0
```

1.3 Aggregate Metrics

Accuracy is included for completeness, but it is not the main metric. For imbalanced vulnerability detection, precision, recall, and F1 are more informative. The key comparison is `test` vs `unseen_test`.

[5]:
```
aggregate_metrics = pea.aggregate_metrics_table(frames)
aggregate_metrics = aggregate_metrics[[
    "support", "positive_rate", "accuracy", "precision", "recall", "f1",
    "TP", "FP", "TN", "FN", "roc_auc", "pr_auc", "brier_score",
]]
display(aggregate_metrics.round(4))
pea.save_dataframe(aggregate_metrics.reset_index(), OUTPUT_DIR /␣
 ↪"aggregate_metrics.csv")

interpretation = pea.interpretation_for_test_vs_unseen(aggregate_metrics)
display(Markdown("### Automatic Interpretation"))
display(Markdown(interpretation))
```

```
             support  positive_rate  accuracy  precision  recall      f1   TP  \
split
test            4727         0.0235    0.9945     0.9048  0.8559  0.8796   95
unseen_test    14430         0.0603    0.9868     0.9292  0.8448  0.8850  735
unseen_val     15487         0.0636    0.8950     0.3599  0.8355  0.5031  823
val             8465         0.0465    0.9894     0.9578  0.8071  0.8760  318

                FP     TN   FN  roc_auc  pr_auc  brier_score
split
test            10   4606   16      NaN     NaN          NaN
unseen_test     56  13504  135      NaN     NaN          NaN
unseen_val    1464  13038  162      NaN     NaN          NaN
val             14   8057   76      NaN     NaN          NaN
```

1.3.1 Automatic Interpretation

Unseen F1 change vs test is +0.005 (small gap). Unseen precision change is +0.024; unseen recall change is -0.011. Generalization to unseen CWE looks stable at the aggregate level.

1.4 Confusion Matrices

Per-split confusion matrices show absolute error counts and row-normalized rates. Row normalization shows what fraction of actual positives/negatives are classified correctly.

[6]:
```
eval_split_names = [s for s in ["val", "test", "unseen_val", "unseen_test"] if␣
 ↪s in frames]
labels = [0, 1]
label_names = ["benign", "vulnerable"]

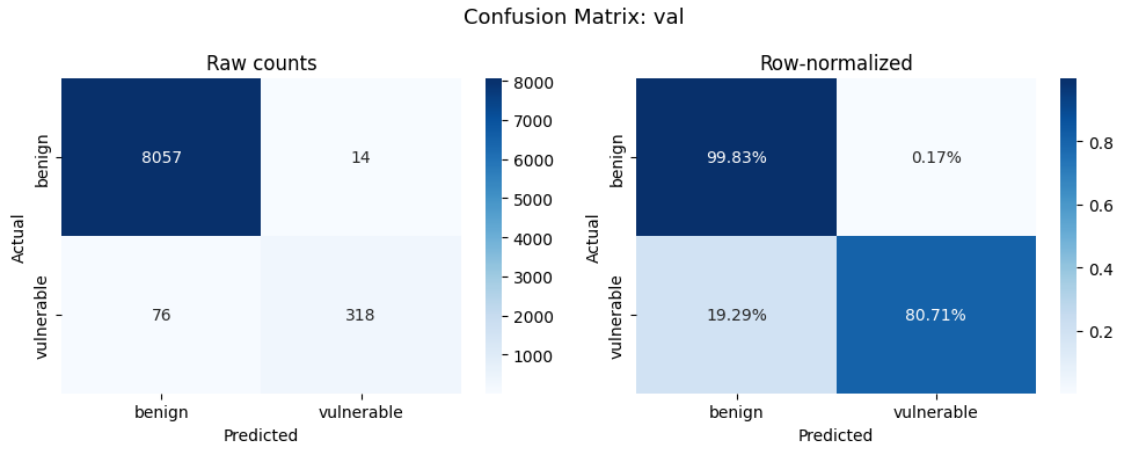
for split_name in eval_split_names:
    df = frames[split_name]
    cm = sk_confusion_matrix(df["y_true"], df["y_pred"], labels=labels)
    cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
    cm_norm = np.nan_to_num(cm_norm)

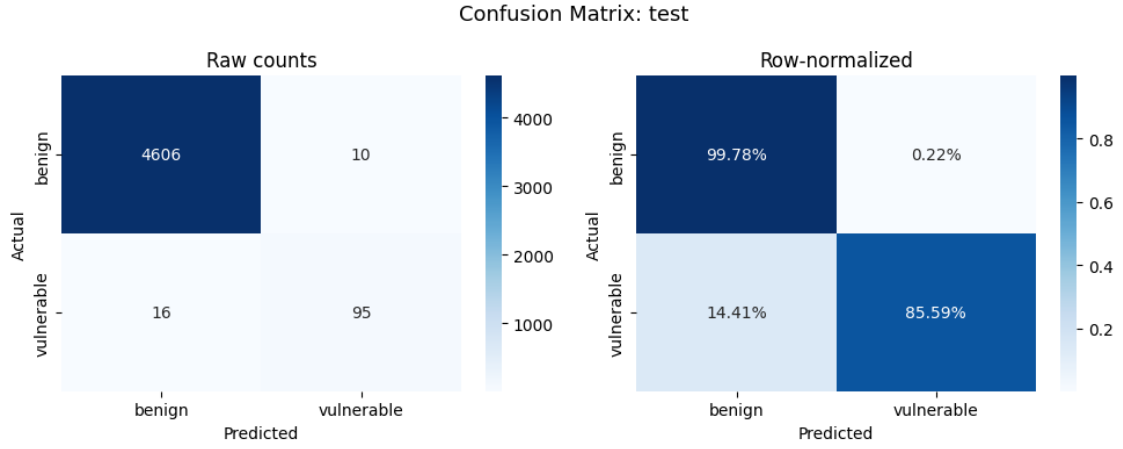
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    fig.suptitle(f"Confusion Matrix: {split_name}", fontsize=13)

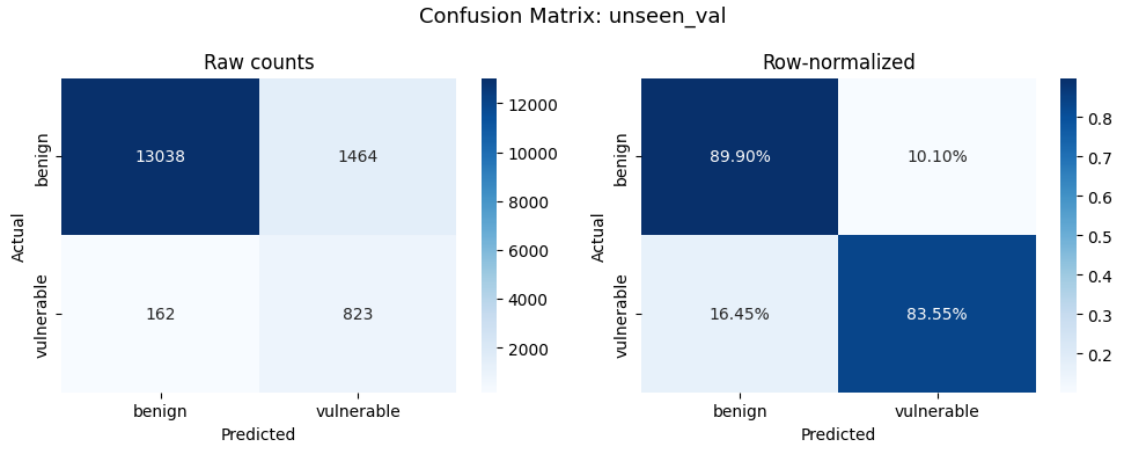
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=label_names,
                yticklabels=label_names, ax=axes[0])
    axes[0].set_title("Raw counts")
    axes[0].set_ylabel("Actual")
    axes[0].set_xlabel("Predicted")

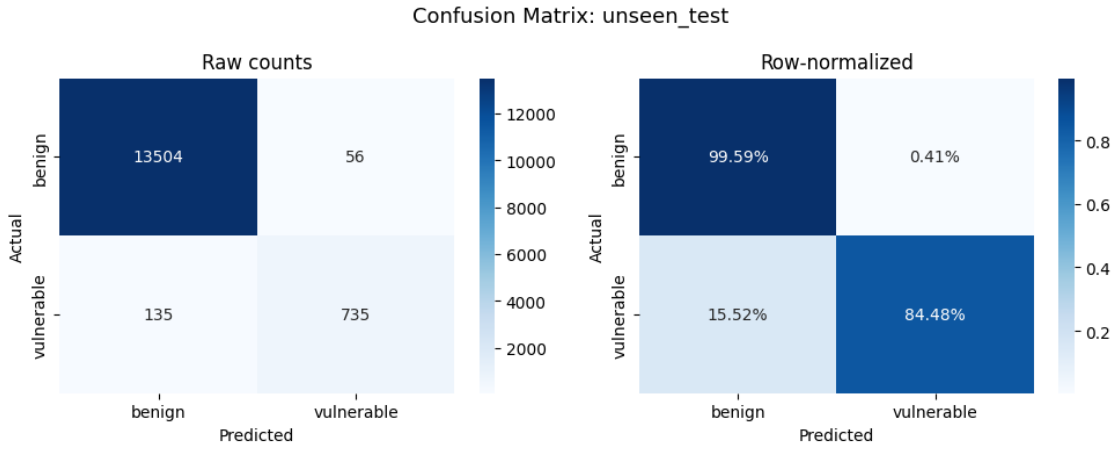
    sns.heatmap(cm_norm, annot=True, fmt=".2%", cmap="Blues",␣
 ↪xticklabels=label_names,
                yticklabels=label_names, ax=axes[1])
    axes[1].set_title("Row-normalized")
    axes[1].set_ylabel("Actual")
    axes[1].set_xlabel("Predicted")

    plt.tight_layout()
    display(fig)
    pea.save_figure(fig, OUTPUT_DIR / f"confusion_matrix_{split_name}.png")
    plt.close(fig)
```


Confusion Matrix: val
Raw counts
Row-normalized
Actual
benign
vulnerable
8057
14
76
318
99.83%
0.17%
19.29%
80.71%
Predicted
benign
vulnerable
8000
7000
6000
5000
4000
3000
2000
1000
0.8
0.6
0.4
0.2


Confusion Matrix: test
Raw counts
Row-normalized
Actual
benign
vulnerable
4606
10
16
95
99.78%
0.22%
14.41%
85.59%
Predicted
benign
vulnerable
4000
3000
2000
1000
0.8
0.6
0.4
0.2


Confusion Matrix: unseen_val
Raw counts
Row-normalized
Actual
benign
vulnerable
13038
1464
162
823
89.90%
10.10%
16.45%
83.55%
Predicted
benign
vulnerable
12000
10000
8000
6000
4000
2000
0.8
0.7
0.6
0.5
0.4
0.3
0.2



1.5 Seen vs Unseen

This section asks whether the gap is mostly in recall, whether the model is more conservative on unseen CWE, and whether false positives or false negatives dominate.

[7]:
```
# Merged eval_df for cross-split analysis (seen vs unseen, error analysis, etc.)
primary_eval_splits = [split for split in ["test", "unseen_test"] if split in␣
 ↪frames]
if not primary_eval_splits:
    primary_eval_splits = [split for split in frames if split != "train"]
eval_df = pd.concat([frames[split].assign(eval_split=split) for split in␣
 ↪primary_eval_splits], ignore_index=True)

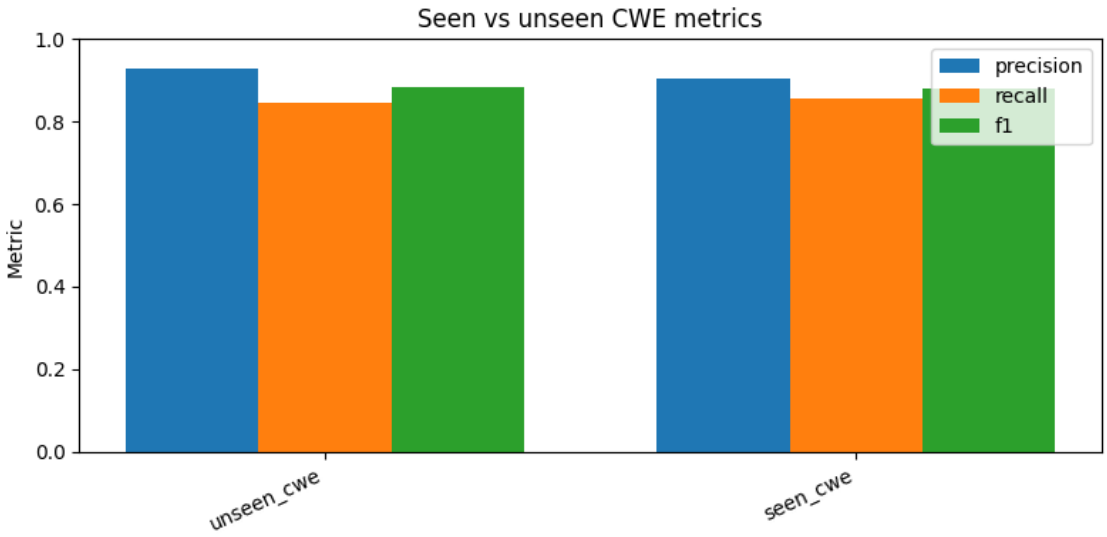
# All non-train splits for per-CWE/per-project breakdowns
all_eval_split_names = [s for s in ["val", "test", "unseen_val", "unseen_test"]␣
 ↪if s in frames]

seen_unseen_metrics = pd.DataFrame()
if "is_unseen_cwe" in eval_df.columns:
    seen_unseen_metrics = pea.grouped_metrics(eval_df, "is_unseen_cwe")
    seen_unseen_metrics["group"] = seen_unseen_metrics["is_unseen_cwe"].map({0:␣
 ↪"seen_cwe", 1: "unseen_cwe"}).fillna("unknown")
    seen_unseen_metrics = seen_unseen_metrics[[
        "group", "support", "positive_rate", "precision", "recall", "f1",␣
 ↪"fp_rate", "fn_rate", "TP", "FP", "TN", "FN"
    ]]
    display(seen_unseen_metrics.round(4))
    pea.save_dataframe(seen_unseen_metrics, OUTPUT_DIR / "seen_unseen_metrics.
 ↪csv")
```

```
    fig = pea.plot_metric_bars(seen_unseen_metrics, "group", ["precision",␣
 ↪"recall", "f1"], "Seen vs unseen CWE metrics")
    display(fig)
    pea.save_figure(fig, OUTPUT_DIR / "seen_vs_unseen_metrics.png")
    plt.close(fig)
else:
    print("TODO: `is_unseen_cwe` not available or not derivable; skipping␣
 ↪grouped seen/unseen analysis.")
```

```
        group  support  positive_rate  precision  recall      f1  fp_rate  \
1  unseen_cwe    14430         0.0603     0.9292  0.8448  0.8850   0.0041
0   seen_cwe     4727         0.0235     0.9048  0.8559  0.8796   0.0022

   fn_rate   TP  FP     TN   FN
1   0.1552  735  56  13504  135
0   0.1441   95  10   4606   16
```



1.6 Unseen Val vs Unseen Test Gap Analysis

This section diagnoses differences between `unseen_val` and `unseen_test`. If there is a large F1 gap, this helps identify whether it comes from project concentration, class distribution differences, or project-specific false positive patterns.

[8]:
```
unseen_splits = {s: frames[s] for s in ["unseen_val", "unseen_test"] if s in␣
 ↪frames}

if len(unseen_splits) < 2:
    print("Skipping unseen gap analysis: need both unseen_val and unseen_test␣
 ↪loaded.")
```

```
else:
    # --- 4.1: Side-by-side aggregate metrics ---
    display(Markdown("### Side-by-side aggregate metrics"))
    unseen_agg = pea.aggregate_metrics_table(unseen_splits)
    unseen_agg_display = unseen_agg[[
        "support", "positive_rate", "accuracy", "precision", "recall", "f1",
        "TP", "FP", "TN", "FN",
    ]]
    display(unseen_agg_display.round(4))
    pea.save_dataframe(unseen_agg_display.reset_index(), OUTPUT_DIR /␣
↪"unseen_gap_aggregate.csv")

    # Delta row
    if {"unseen_val", "unseen_test"}.issubset(unseen_agg.index):
        gap_rows = []
        for m in ["accuracy", "precision", "recall", "f1", "positive_rate"]:
            gap_rows.append({
                "metric": m,
                "unseen_val": unseen_agg.loc["unseen_val", m],
                "unseen_test": unseen_agg.loc["unseen_test", m],
                "delta (val - test)": unseen_agg.loc["unseen_val", m] -␣
↪unseen_agg.loc["unseen_test", m],
            })
        display(Markdown("### Metric deltas (unseen_val - unseen_test)"))
        display(pd.DataFrame(gap_rows).round(4))

    # --- 4.2: Per-project FP diagnostics ---
    display(Markdown("### Per-project FP diagnostics"))
    for split_name, split_df in unseen_splits.items():
        pp = pea.per_project_metrics(split_df)
        if pp.empty:
            print(f"No project column in {split_name}.")
            continue
        display(Markdown(f"#### {split_name}: projects by FP count"))
        display(pp.sort_values(["FP", "support"], ascending=[False, False]).
↪head(15).round(4))

    # --- 4.3: Project concentration ---
    display(Markdown("### Project concentration"))
    concentration_rows = []
    for split_name, split_df in unseen_splits.items():
        if "project" not in split_df.columns:
            continue
        proj_counts = split_df["project"].value_counts()
        total = len(split_df)
        top3_share = proj_counts.head(3).sum() / total
        top5_share = proj_counts.head(5).sum() / total
```

```
        n_projects = proj_counts.shape[0]
        concentration_rows.append({
            "split": split_name,
            "n_projects": n_projects,
            "top_3_row_share": top3_share,
            "top_5_row_share": top5_share,
            "largest_project": proj_counts.index[0],
            "largest_project_share": proj_counts.iloc[0] / total,
        })
    if concentration_rows:
        display(pd.DataFrame(concentration_rows).round(4))

    # --- 4.4: Class distribution comparison ---
    display(Markdown("### Positive rate per project across unseen splits"))
    pos_rate_frames = []
    for split_name, split_df in unseen_splits.items():
        if "project" not in split_df.columns:
            continue
        pr = split_df.groupby("project").agg(
            support=("y_true", "size"),
            positive_rate=("y_true", "mean"),
            n_positive=("y_true", "sum"),
        ).reset_index()
        pr["split"] = split_name
        pos_rate_frames.append(pr)
    if pos_rate_frames:
        pos_rate_all = pd.concat(pos_rate_frames, ignore_index=True)
        pos_rate_pivot = pos_rate_all.pivot_table(
            index="project", columns="split",
            values=["positive_rate", "support"], aggfunc="first",
        )
        pos_rate_pivot.columns = [f"{col[1]}_{col[0]}" for col in␣
↪pos_rate_pivot.columns]
        pos_rate_pivot = pos_rate_pivot.fillna(0)
        if "unseen_val_positive_rate" in pos_rate_pivot.columns and␣
↪"unseen_test_positive_rate" in pos_rate_pivot.columns:
            pos_rate_pivot["pos_rate_diff"] =␣
↪pos_rate_pivot["unseen_val_positive_rate"] -␣
↪pos_rate_pivot["unseen_test_positive_rate"]
            display(pos_rate_pivot.sort_values("pos_rate_diff",␣
↪ascending=False).round(4))
        else:
            display(pos_rate_pivot.round(4))
        pea.save_dataframe(pos_rate_pivot.reset_index(), OUTPUT_DIR /␣
↪"unseen_gap_positive_rate_by_project.csv")
```

1.6.1 Side-by-side aggregate metrics

split	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	FN
unseen_test	14430	0.0603	0.9868	0.9292	0.8448	0.8850	735	56	13504	135
unseen_val	15487	0.0636	0.8950	0.3599	0.8355	0.5031	823	1464	13038	162

1.6.2 Metric deltas (unseen_val - unseen_test)

	metric	unseen_val	unseen_test	delta (val - test)
0	accuracy	0.8950	0.9868	-0.0918
1	precision	0.3599	0.9292	-0.5693
2	recall	0.8355	0.8448	-0.0093
3	f1	0.5031	0.8850	-0.3820
4	positive_rate	0.0636	0.0603	0.0033

1.6.3 Per-project FP diagnostics

unseen_val: projects by FP count

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project
0	5287	0.0758	0.7314	0.2117	0.9327	0.3450	374	1393	3493	27	0.2851	0.0673	NaN	NaN	NaN	Android
1	2119	0.0835	0.9547	0.7872	0.6271	0.6981	111	30	1912	66	0.0154	0.3729	NaN	NaN	NaN	ImageMagick
6	697	0.1363	0.9770	0.8692	0.9789	0.9208	93	14	588	2	0.0233	0.0211	NaN	NaN	NaN	jasper
15	3042	0.0381	0.9915	0.9245	0.8448	0.8829	98	8	2918	18	0.0027	0.1552	NaN	NaN	NaN	qemu
16	1965	0.0356	0.9878	0.8710	0.7714	0.8182	54	8	1887	16	0.0042	0.2286	NaN	NaN	NaN	savannah
13	1902	0.0526	0.9811	0.9000	0.7200	0.8000	72	8	1794	28	0.0044	0.2800	NaN	NaN	NaN	php
7	51	0.0588	0.9608	0.6000	1.0000	0.7500	3	2	46	0	0.0417	0.0000	NaN	NaN	NaN	gpac
10	16	0.0625	0.9375	0.5000	1.0000	0.6667	1	1	14	0	0.0667	0.0000	NaN	NaN	NaN	imagemagick
5	89	0.0449	0.9888	1.0000	0.7500	0.8571	3	0	85	1	0.0000	0.2500	NaN	NaN	NaN	file
11	81	0.0123	1.0000	1.0000	1.0000	1.0000	1	0	80	0	0.0000	0.0000	NaN	NaN	NaN	openssl
3	53	0.1132	0.9811	1.0000	0.8333	0.9091	5	0	47	1	0.0000	0.1667	NaN	NaN	NaN	curl
18	40	0.0500	0.9750	1.0000	0.5000	0.6667	1	0	38	1	0.0000	0.5000	NaN	NaN	NaN	wireshark
2	36	0.0556	1.0000	1.0000	1.0000	1.0000	2	0	34	0	0.0000	0.0000	NaN	NaN	NaN	ffmpeg
14	35	0.0857	0.9714	1.0000	0.6667	0.8000	2	0	33	1	0.0000	0.3333	NaN	NaN	NaN	linux
8	30	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	30	0	0.0000	0.0000	NaN	NaN	NaN	gnutls

7	46	0	0.0417	0.0000	NaN	NaN	NaN	libvips
10	14	0	0.0667	0.0000	NaN	NaN	NaN	netdata
5	85	1	0.0000	0.2500	NaN	NaN	NaN	gstreamer
11	80	0	0.0000	0.0000	NaN	NaN	NaN	openvpn
3	47	1	0.0000	0.1667	NaN	NaN	NaN	abrt
18	38	1	0.0000	0.5000	NaN	NaN	NaN	zfs
2	34	0	0.0000	0.0000	NaN	NaN	NaN	NetworkManager
14	32	1	0.0000	0.3333	NaN	NaN	NaN	polkit
8	30	0	0.0000	NaN	NaN	NaN	NaN	lynx-snapshots

	vulnerable_examples	benign_examples	unique_cwes
0	401	4886	22
1	177	1942	21
6	95	602	8
15	116	2926	21
16	70	1895	18
13	100	1802	16
7	3	48	2
10	1	15	1
5	4	85	4
11	1	80	2
3	6	47	2
18	2	38	1
2	2	34	2
14	3	32	1
8	0	30	1

unseen_test: projects by FP count

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	\
9	212	0.0330	0.9575	0.4286	0.8571	0.5714	6	8	
21	1419	0.0648	0.9803	0.9103	0.7717	0.8353	71	7	
23	1101	0.0509	0.9855	0.8846	0.8214	0.8519	46	6	
4	1737	0.0345	0.9937	0.9153	0.9000	0.9076	54	5	
22	1456	0.1422	0.9794	0.9733	0.8792	0.9239	182	5	
0	1695	0.0490	0.9876	0.9429	0.7952	0.8627	66	4	
1	337	0.0475	0.9703	0.7143	0.6250	0.6667	10	4	
3	233	0.0386	0.9785	0.6667	0.8889	0.7619	8	4	
29	102	0.0098	0.9706	0.2500	1.0000	0.4000	1	3	
27	401	0.0100	0.9950	0.6667	1.0000	0.8000	4	2	
19	257	0.0195	0.9844	0.6000	0.6000	0.6000	3	2	
24	138	0.0507	0.9855	0.7778	1.0000	0.8750	7	2	
12	668	0.0299	0.9955	0.9474	0.9000	0.9231	18	1	
26	316	0.0759	0.9684	0.9375	0.6250	0.7500	15	1	
18	315	0.0190	0.9968	0.8571	1.0000	0.9231	6	1	

	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project	\
9	197	1	0.0390	0.1429	NaN	NaN	NaN	kde	

21	1320	21	0.0053	0.2283	NaN	NaN	NaN	openssl
23	1039	10	0.0057	0.1786	NaN	NaN	NaN	radare2
4	1672	6	0.0030	0.1000	NaN	NaN	NaN	ghostscript
22	1244	25	0.0040	0.1208	NaN	NaN	NaN	php-src
0	1608	17	0.0025	0.2048	NaN	NaN	NaN	FFmpeg
1	317	6	0.0125	0.3750	NaN	NaN	NaN	ImageMagick6
3	220	1	0.0179	0.1111	NaN	NaN	NaN	exempi
29	98	0	0.0297	0.0000	NaN	NaN	NaN	wolfssl
27	395	0	0.0050	0.0000	NaN	NaN	NaN	tor
19	250	2	0.0079	0.4000	NaN	NaN	NaN	oniguruma
24	129	0	0.0153	0.0000	NaN	NaN	NaN	raptor
12	647	2	0.0015	0.1000	NaN	NaN	NaN	libarchive
26	291	9	0.0034	0.3750	NaN	NaN	NaN	src
18	308	0	0.0032	0.0000	NaN	NaN	NaN	mruby

	vulnerable_examples	benign_examples	unique_cwes
9	7	205	4
21	92	1327	19
23	56	1045	6
4	60	1677	11
22	207	1249	17
0	83	1612	15
1	16	321	6
3	9	224	6
29	1	101	1
27	4	397	4
19	5	252	5
24	7	131	1
12	20	648	11
26	24	292	7
18	6	309	5

1.6.4 Project concentration

	split	n_projects	top_3_row_share	top_5_row_share	largest_project	\
0	unseen_val	19	0.6746	0.9243	Android	
1	unseen_test	31	0.3387	0.5134	ghostscript	

	largest_project_share
0	0.3414
1	0.1204

1.6.5 Positive rate per project across unseen splits

project	unseen_test_positive_rate	unseen_val_positive_rate	\
jasper	0.0	0.1363	
acpica	0.0	0.1333	

abrt	0.0	0.1132
pam-u2f	0.0	0.1
polkit	0.0	0.0857
ImageMagick	0.0	0.0835
yubico-pam	0.0	0.0833
Android	0.0	0.0758
netdata	0.0	0.0625
libvips	0.0	0.0588
NetworkManager	0.0	0.0556
php	0.0	0.0526
zfs	0.0	0.05
gstreamer	0.0	0.0449
qemu	0.0	0.0381
savannah	0.0	0.0356
openvpn	0.0	0.0123
mongoose-os	0.0	0.0
lynx-snapshots	0.0	0.0
udisks	0.0074	0.0
wolfssl	0.0098	0.0
tor	0.01	0.0
httpd	0.0118	0.0
haproxy	0.0118	0.0
mruby	0.019	0.0
oniguruma	0.0195	0.0
libreport	0.0278	0.0
libarchive	0.0299	0.0
kde	0.033	0.0
ghostscript	0.0345	0.0
exempi	0.0386	0.0
keepalived	0.0387	0.0
libcomps	0.04	0.0
ImageMagick6	0.0475	0.0
FFmpeg	0.049	0.0
gnupg	0.049	0.0
raptor	0.0507	0.0
radare2	0.0509	0.0
curl	0.0558	0.0
irssi	0.0588	0.0
miniupnp	0.0602	0.0
openssl	0.0648	0.0
libxkbcommon	0.0738	0.0
src	0.0759	0.0
yara	0.0766	0.0
mbedtls	0.0773	0.0
samba	0.0818	0.0
openjpeg	0.1364	0.0
php-src	0.1422	0.0
krb5	0.2008	0.0

project	unseen_test_support	unseen_val_support	pos_rate_diff
jasper	0	697	0.1363
acpica	0	15	0.1333
abrt	0	53	0.1132
pam-u2f	0	10	0.1
polkit	0	35	0.0857
ImageMagick	0	2119	0.0835
yubico-pam	0	12	0.0833
Android	0	5287	0.0758
netdata	0	16	0.0625
libvips	0	51	0.0588
NetworkManager	0	36	0.0556
php	0	1902	0.0526
zfs	0	40	0.05
gstreamer	0	89	0.0449
qemu	0	3042	0.0381
savannah	0	1965	0.0356
openvpn	0	81	0.0123
mongoose-os	0	7	0.0
lynx-snapshots	0	30	0.0
udisks	272	0	-0.0074
wolfssl	102	0	-0.0098
tor	401	0	-0.01
httpd	509	0	-0.0118
haproxy	339	0	-0.0118
mruby	315	0	-0.019
oniguruma	257	0	-0.0195
libreport	108	0	-0.0278
libarchive	668	0	-0.0299
kde	212	0	-0.033
ghostscript	1737	0	-0.0345
exempi	233	0	-0.0386
keepalived	388	0	-0.0387
libcomps	100	0	-0.04
ImageMagick6	337	0	-0.0475
FFmpeg	1695	0	-0.049
gnupg	204	0	-0.049
raptor	138	0	-0.0507
radare2	1101	0	-0.0509
curl	215	0	-0.0558
irssi	119	0	-0.0588
miniupnp	133	0	-0.0602
openssl	1419	0	-0.0648
libxkbcommon	122	0	-0.0738
src	316	0	-0.0759
yara	209	0	-0.0766

mbedtls	194	0	-0.0773
samba	440	0	-0.0818
openjpeg	198	0	-0.1364
php-src	1456	0	-0.1422
krb5	493	0	-0.2008

1.7 Per-CWE Analysis

Aggregate metrics can hide instability in rare classes. This section is one of the main decision points for future runs.

[9]:
```
per_cwe_by_split = {}
for split_name in all_eval_split_names:
    split_df = frames[split_name]
    per_cwe = pea.per_cwe_metrics(split_df)
    if per_cwe.empty:
        print(f"TODO: `cwe_id` is missing in {split_name}; skipping per-CWE␣
↪analysis for this split.")
        continue
    per_cwe_by_split[split_name] = per_cwe
    pea.save_dataframe(per_cwe, OUTPUT_DIR / f"per_cwe_metrics_{split_name}.
↪csv")

    display(Markdown(f"### Per-CWE: {split_name}"))
    display(Markdown(f"#### Worst CWEs by F1 ({split_name})"))
    display(per_cwe.sort_values(["f1", "support"], ascending=[True, False]).
↪head(15).round(4))
    display(Markdown(f"#### Worst CWEs by recall ({split_name})"))
    display(per_cwe.sort_values(["recall", "support"], ascending=[True, False]).
↪head(15).round(4))
    display(Markdown(f"#### Best CWEs ({split_name})"))
    display(per_cwe.sort_values(["f1", "support"], ascending=[False, False]).
↪head(15).round(4))

    if "is_unseen_cwe" in per_cwe.columns:
        display(Markdown(f"#### Seen CWE table ({split_name})"))
        display(per_cwe[per_cwe["is_unseen_cwe"] == 0].sort_values(["f1",␣
↪"support"], ascending=[True, False]).head(15).round(4))
        display(Markdown(f"#### Unseen CWE table ({split_name})"))
        display(per_cwe[per_cwe["is_unseen_cwe"] == 1].sort_values(["f1",␣
↪"support"], ascending=[True, False]).head(15).round(4))

    hue_col = "is_unseen_cwe" if "is_unseen_cwe" in per_cwe.columns else None
    fig = pea.plot_support_scatter(per_cwe, "support", "f1", f"CWE support vs␣
↪F1 ({split_name})", hue_col=hue_col)
    display(fig)
    pea.save_figure(fig, OUTPUT_DIR / f"per_cwe_support_vs_f1_{split_name}.png")
```

```
    plt.close(fig)

    low_support = per_cwe[per_cwe["support"] < 10]
    if not low_support.empty:
        display(Markdown(f"**Caution:** {len(low_support)} CWE rows have␣
↪support < 10 in {split_name}."))
```

1.7.1 Per-CWE: val

Worst CWEs by F1 (val)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
15	87	0.0460	0.9655	1.0000	0.2500	0.4000	1	0	83	
30	9	0.2222	0.7778	0.5000	0.5000	0.5000	1	1	6	
22	120	0.0083	0.9917	0.5000	1.0000	0.6667	1	1	118	
4	364	0.0824	0.9643	0.9474	0.6000	0.7347	18	1	333	
8	132	0.0606	0.9697	0.7000	0.8750	0.7778	7	3	121	
32	310	0.0226	0.9935	1.0000	0.7143	0.8333	5	0	303	
20	1753	0.0434	0.9875	0.9500	0.7500	0.8382	57	3	1674	
5	392	0.0918	0.9745	1.0000	0.7222	0.8387	26	0	356	
18	896	0.0670	0.9821	0.9783	0.7500	0.8491	45	1	835	
17	852	0.0246	0.9930	0.8947	0.8095	0.8500	17	2	829	
26	58	0.0517	0.9828	0.7500	1.0000	0.8571	3	1	54	
33	491	0.0204	0.9959	1.0000	0.8000	0.8889	8	0	481	
10	250	0.0600	0.9880	0.9286	0.8667	0.8966	13	1	234	
19	212	0.0425	0.9953	1.0000	0.8889	0.9412	8	0	203	
1	556	0.0558	0.9946	1.0000	0.9032	0.9492	28	0	525	

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
15	3	0.0000	0.7500	NaN	NaN	NaN	CWE-369	
30	1	0.1429	0.5000	NaN	NaN	NaN	CWE-763	
22	0	0.0084	0.0000	NaN	NaN	NaN	CWE-59	
4	12	0.0030	0.4000	NaN	NaN	NaN	CWE-17	
8	1	0.0242	0.1250	NaN	NaN	NaN	CWE-254	
32	2	0.0000	0.2857	NaN	NaN	NaN	CWE-772	
20	19	0.0018	0.2500	NaN	NaN	NaN	CWE-476	
5	10	0.0000	0.2778	NaN	NaN	NaN	CWE-19	
18	15	0.0012	0.2500	NaN	NaN	NaN	CWE-404	
17	4	0.0024	0.1905	NaN	NaN	NaN	CWE-400	
26	0	0.0182	0.0000	NaN	NaN	NaN	CWE-732	
33	2	0.0000	0.2000	NaN	NaN	NaN	CWE-787	
10	2	0.0043	0.1333	NaN	NaN	NaN	CWE-284	
19	1	0.0000	0.1111	NaN	NaN	NaN	CWE-415	
1	3	0.0000	0.0968	NaN	NaN	NaN	CWE-125	

	vulnerable_examples	benign_examples	is_unseen_cwe
15	4	83	0
30	2	7	0

22	1	119	0
4	30	334	0
8	8	124	0
32	7	303	0
20	76	1677	0
5	36	356	0
18	60	836	0
17	21	831	0
26	3	55	0
33	10	481	0
10	15	235	0
19	9	203	0
1	31	525	0

Worst CWEs by recall (val)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
15	87	0.0460	0.9655	1.0000	0.2500	0.4000	1	0	83	
30	9	0.2222	0.7778	0.5000	0.5000	0.5000	1	1	6	
4	364	0.0824	0.9643	0.9474	0.6000	0.7347	18	1	333	
32	310	0.0226	0.9935	1.0000	0.7143	0.8333	5	0	303	
5	392	0.0918	0.9745	1.0000	0.7222	0.8387	26	0	356	
20	1753	0.0434	0.9875	0.9500	0.7500	0.8382	57	3	1674	
18	896	0.0670	0.9821	0.9783	0.7500	0.8491	45	1	835	
33	491	0.0204	0.9959	1.0000	0.8000	0.8889	8	0	481	
17	852	0.0246	0.9930	0.8947	0.8095	0.8500	17	2	829	
10	250	0.0600	0.9880	0.9286	0.8667	0.8966	13	1	234	
8	132	0.0606	0.9697	0.7000	0.8750	0.7778	7	3	121	
19	212	0.0425	0.9953	1.0000	0.8889	0.9412	8	0	203	
1	556	0.0558	0.9946	1.0000	0.9032	0.9492	28	0	525	
6	477	0.0314	0.9979	1.0000	0.9333	0.9655	14	0	462	
13	492	0.0325	1.0000	1.0000	1.0000	1.0000	16	0	476	

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
15	3	0.0000	0.7500	NaN	NaN	NaN	CWE-369	
30	1	0.1429	0.5000	NaN	NaN	NaN	CWE-763	
4	12	0.0030	0.4000	NaN	NaN	NaN	CWE-17	
32	2	0.0000	0.2857	NaN	NaN	NaN	CWE-772	
5	10	0.0000	0.2778	NaN	NaN	NaN	CWE-19	
20	19	0.0018	0.2500	NaN	NaN	NaN	CWE-476	
18	15	0.0012	0.2500	NaN	NaN	NaN	CWE-404	
33	2	0.0000	0.2000	NaN	NaN	NaN	CWE-787	
17	4	0.0024	0.1905	NaN	NaN	NaN	CWE-400	
10	2	0.0043	0.1333	NaN	NaN	NaN	CWE-284	
8	1	0.0242	0.1250	NaN	NaN	NaN	CWE-254	
19	1	0.0000	0.1111	NaN	NaN	NaN	CWE-415	
1	3	0.0000	0.0968	NaN	NaN	NaN	CWE-125	
6	1	0.0000	0.0667	NaN	NaN	NaN	CWE-190	

```
13   0   0.0000   0.0000      NaN     NaN          NaN  CWE-310
```

```
    vulnerable_examples  benign_examples  is_unseen_cwe
15                    4               83              0
30                    2                7              0
4                    30              334              0
32                    7              303              0
5                    36              356              0
20                   76             1677              0
18                   60              836              0
33                   10              481              0
17                   21              831              0
10                   15              235              0
8                     8              124              0
19                    9              203              0
1                    31              525              0
6                    15              462              0
13                   16              476              0
```

Best CWEs (val)

```
    support  positive_rate  accuracy  precision  recall   f1  TP  FP   TN  FN  \
13      492         0.0325       1.0        1.0     1.0  1.0  16   0  476   0
16      264         0.0303       1.0        1.0     1.0  1.0   8   0  256   0
12      142         0.0211       1.0        1.0     1.0  1.0   3   0  139   0
11      118         0.1186       1.0        1.0     1.0  1.0  14   0  104   0
23       82         0.0122       1.0        1.0     1.0  1.0   1   0   81   0
2        58         0.0517       1.0        1.0     1.0  1.0   3   0   55   0
3        58         0.0172       1.0        1.0     1.0  1.0   1   0   57   0
9        52         0.0192       1.0        1.0     1.0  1.0   1   0   51   0
7        47         0.0213       1.0        1.0     1.0  1.0   1   0   46   0
25       33         0.0606       1.0        1.0     1.0  1.0   2   0   31   0
34       28         0.0714       1.0        1.0     1.0  1.0   2   0   26   0
0        25         0.1200       1.0        1.0     1.0  1.0   3   0   22   0
24       24         0.0417       1.0        1.0     1.0  1.0   1   0   23   0
29       23         0.0870       1.0        1.0     1.0  1.0   2   0   21   0
27       18         0.0556       1.0        1.0     1.0  1.0   1   0   17   0
```

```
    fp_rate  fn_rate  roc_auc  pr_auc  brier_score   cwe_id  \
13      0.0      0.0      NaN     NaN          NaN  CWE-310
16      0.0      0.0      NaN     NaN          NaN  CWE-388
12      0.0      0.0      NaN     NaN          NaN  CWE-287
11      0.0      0.0      NaN     NaN          NaN  CWE-285
23      0.0      0.0      NaN     NaN          NaN  CWE-617
2       0.0      0.0      NaN     NaN          NaN  CWE-129
3       0.0      0.0      NaN     NaN          NaN   CWE-16
9       0.0      0.0      NaN     NaN          NaN  CWE-269
7       0.0      0.0      NaN     NaN          NaN  CWE-191
```

25	0.0	0.0	NaN	NaN	NaN	CWE-704
34	0.0	0.0	NaN	NaN	NaN	CWE-835
0	0.0	0.0	NaN	NaN	NaN	CWE-120
24	0.0	0.0	NaN	NaN	NaN	CWE-665
29	0.0	0.0	NaN	NaN	NaN	CWE-755
27	0.0	0.0	NaN	NaN	NaN	CWE-74

	vulnerable_examples	benign_examples	is_unseen_cwe
13	16	476	0
16	8	256	0
12	3	139	0
11	14	104	0
23	1	81	0
2	3	55	0
3	1	57	0
9	1	51	0
7	1	46	0
25	2	31	0
34	2	26	0
0	3	22	0
24	1	23	0
29	2	21	0
27	1	17	0

Seen CWE table (val)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
15	87	0.0460	0.9655	1.0000	0.2500	0.4000	1	0	83	
30	9	0.2222	0.7778	0.5000	0.5000	0.5000	1	1	6	
22	120	0.0083	0.9917	0.5000	1.0000	0.6667	1	1	118	
4	364	0.0824	0.9643	0.9474	0.6000	0.7347	18	1	333	
8	132	0.0606	0.9697	0.7000	0.8750	0.7778	7	3	121	
32	310	0.0226	0.9935	1.0000	0.7143	0.8333	5	0	303	
20	1753	0.0434	0.9875	0.9500	0.7500	0.8382	57	3	1674	
5	392	0.0918	0.9745	1.0000	0.7222	0.8387	26	0	356	
18	896	0.0670	0.9821	0.9783	0.7500	0.8491	45	1	835	
17	852	0.0246	0.9930	0.8947	0.8095	0.8500	17	2	829	
26	58	0.0517	0.9828	0.7500	1.0000	0.8571	3	1	54	
33	491	0.0204	0.9959	1.0000	0.8000	0.8889	8	0	481	
10	250	0.0600	0.9880	0.9286	0.8667	0.8966	13	1	234	
19	212	0.0425	0.9953	1.0000	0.8889	0.9412	8	0	203	
1	556	0.0558	0.9946	1.0000	0.9032	0.9492	28	0	525	

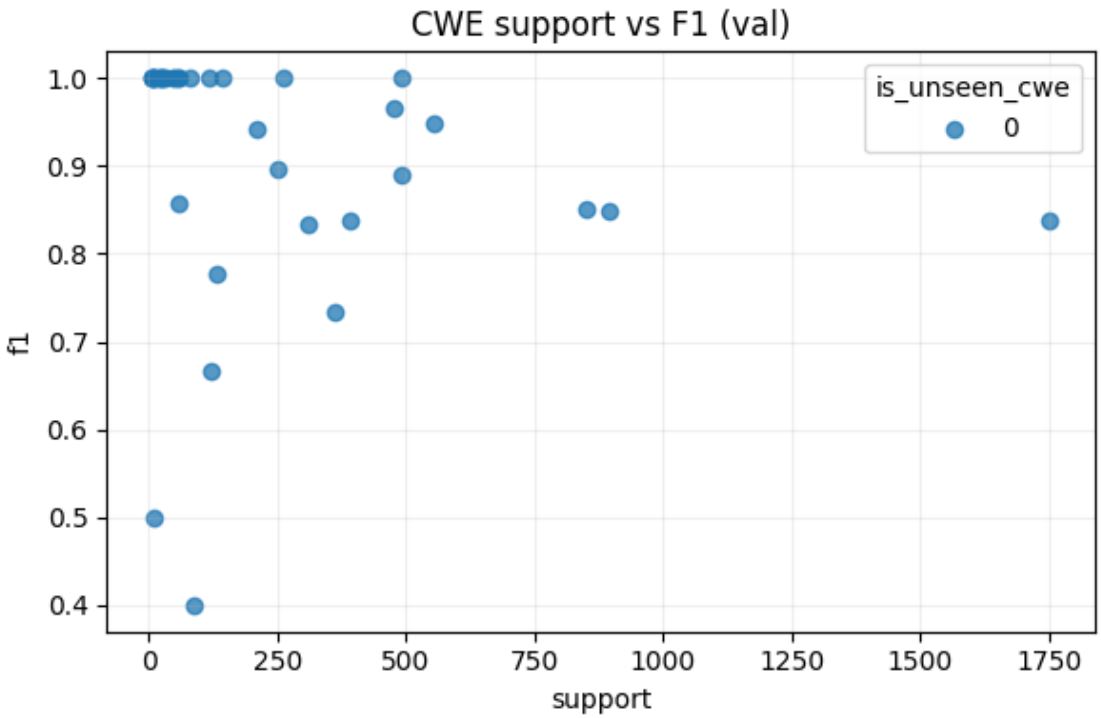
	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
15	3	0.0000	0.7500	NaN	NaN	NaN	CWE-369	
30	1	0.1429	0.5000	NaN	NaN	NaN	CWE-763	
22	0	0.0084	0.0000	NaN	NaN	NaN	CWE-59	
4	12	0.0030	0.4000	NaN	NaN	NaN	CWE-17	

8	1	0.0242	0.1250	NaN	NaN	NaN	CWE-254
32	2	0.0000	0.2857	NaN	NaN	NaN	CWE-772
20	19	0.0018	0.2500	NaN	NaN	NaN	CWE-476
5	10	0.0000	0.2778	NaN	NaN	NaN	CWE-19
18	15	0.0012	0.2500	NaN	NaN	NaN	CWE-404
17	4	0.0024	0.1905	NaN	NaN	NaN	CWE-400
26	0	0.0182	0.0000	NaN	NaN	NaN	CWE-732
33	2	0.0000	0.2000	NaN	NaN	NaN	CWE-787
10	2	0.0043	0.1333	NaN	NaN	NaN	CWE-284
19	1	0.0000	0.1111	NaN	NaN	NaN	CWE-415
1	3	0.0000	0.0968	NaN	NaN	NaN	CWE-125

	vulnerable_examples	benign_examples	is_unseen_cwe
15	4	83	0
30	2	7	0
22	1	119	0
4	30	334	0
8	8	124	0
32	7	303	0
20	76	1677	0
5	36	356	0
18	60	836	0
17	21	831	0
26	3	55	0
33	10	481	0
10	15	235	0
19	9	203	0
1	31	525	0

Unseen CWE table (val)

```
Empty DataFrame
Columns: [support, positive_rate, accuracy, precision, recall, f1, TP, FP, TN,␣
 ↪FN, fp_rate, fn_rate, roc_auc, pr_auc, brier_score, cwe_id,␣
 ↪vulnerable_examples, benign_examples, is_unseen_cwe]
Index: []
```



Caution: 4 CWE rows have support < 10 in val.

1.7.2 Per-CWE: test

Worst CWEs by F1 (test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
8	3	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	3	
10	3	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	3	
1	956	0.0063	0.9969	0.8000	0.6667	0.7273	4	1	949	
14	199	0.1106	0.9497	0.8333	0.6818	0.7500	15	3	174	
2	743	0.0283	0.9933	0.9444	0.8095	0.8718	17	1	721	
5	667	0.0165	0.9955	0.7857	1.0000	0.8800	11	3	653	
0	571	0.0263	0.9965	0.9333	0.9333	0.9333	14	1	555	
4	719	0.0376	0.9958	0.9615	0.9259	0.9434	25	1	691	
12	638	0.0031	1.0000	1.0000	1.0000	1.0000	2	0	636	
7	113	0.0088	1.0000	1.0000	1.0000	1.0000	1	0	112	
9	51	0.0196	1.0000	1.0000	1.0000	1.0000	1	0	50	
6	34	0.0294	1.0000	1.0000	1.0000	1.0000	1	0	33	
3	27	0.0741	1.0000	1.0000	1.0000	1.0000	2	0	25	
13	2	0.5000	1.0000	1.0000	1.0000	1.0000	1	0	1	
11	1	1.0000	1.0000	1.0000	1.0000	1.0000	1	0	0	

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
8	0	0.0000	NaN	NaN	NaN	NaN	CWE-399	

10	0	0.0000	NaN	NaN	NaN	NaN	CWE-476
1	2	0.0011	0.3333	NaN	NaN	NaN	CWE-125
14	7	0.0169	0.3182	NaN	NaN	NaN	CWE-787
2	4	0.0014	0.1905	NaN	NaN	NaN	CWE-189
5	0	0.0046	0.0000	NaN	NaN	NaN	CWE-269
0	1	0.0018	0.0667	NaN	NaN	NaN	CWE-119
4	2	0.0014	0.0741	NaN	NaN	NaN	CWE-20
12	0	0.0000	0.0000	NaN	NaN	NaN	CWE-617
7	0	0.0000	0.0000	NaN	NaN	NaN	CWE-369
9	0	0.0000	0.0000	NaN	NaN	NaN	CWE-400
6	0	0.0000	0.0000	NaN	NaN	NaN	CWE-362
3	0	0.0000	0.0000	NaN	NaN	NaN	CWE-190
13	0	0.0000	0.0000	NaN	NaN	NaN	CWE-665
11	0	NaN	0.0000	NaN	NaN	NaN	CWE-59

	vulnerable_examples	benign_examples	is_unseen_cwe
8	0	3	0
10	0	3	0
1	6	950	0
14	22	177	0
2	21	722	0
5	11	656	0
0	15	556	0
4	27	692	0
12	2	636	0
7	1	112	0
9	1	50	0
6	1	33	0
3	2	25	0
13	1	1	0
11	1	0	0

Worst CWEs by recall (test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
8	3	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	3	
10	3	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	3	
1	956	0.0063	0.9969	0.8000	0.6667	0.7273	4	1	949	
14	199	0.1106	0.9497	0.8333	0.6818	0.7500	15	3	174	
2	743	0.0283	0.9933	0.9444	0.8095	0.8718	17	1	721	
4	719	0.0376	0.9958	0.9615	0.9259	0.9434	25	1	691	
0	571	0.0263	0.9965	0.9333	0.9333	0.9333	14	1	555	
5	667	0.0165	0.9955	0.7857	1.0000	0.8800	11	3	653	
12	638	0.0031	1.0000	1.0000	1.0000	1.0000	2	0	636	
7	113	0.0088	1.0000	1.0000	1.0000	1.0000	1	0	112	
9	51	0.0196	1.0000	1.0000	1.0000	1.0000	1	0	50	
6	34	0.0294	1.0000	1.0000	1.0000	1.0000	1	0	33	
3	27	0.0741	1.0000	1.0000	1.0000	1.0000	2	0	25	

13	2	0.5000	1.0000	1.0000	1.0000	1.0000	1	0	1
11	1	1.0000	1.0000	1.0000	1.0000	1.0000	1	0	0

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
8	0	0.0000	NaN	NaN	NaN	NaN	CWE-399	
10	0	0.0000	NaN	NaN	NaN	NaN	CWE-476	
1	2	0.0011	0.3333	NaN	NaN	NaN	CWE-125	
14	7	0.0169	0.3182	NaN	NaN	NaN	CWE-787	
2	4	0.0014	0.1905	NaN	NaN	NaN	CWE-189	
4	2	0.0014	0.0741	NaN	NaN	NaN	CWE-20	
0	1	0.0018	0.0667	NaN	NaN	NaN	CWE-119	
5	0	0.0046	0.0000	NaN	NaN	NaN	CWE-269	
12	0	0.0000	0.0000	NaN	NaN	NaN	CWE-617	
7	0	0.0000	0.0000	NaN	NaN	NaN	CWE-369	
9	0	0.0000	0.0000	NaN	NaN	NaN	CWE-400	
6	0	0.0000	0.0000	NaN	NaN	NaN	CWE-362	
3	0	0.0000	0.0000	NaN	NaN	NaN	CWE-190	
13	0	0.0000	0.0000	NaN	NaN	NaN	CWE-665	
11	0	NaN	0.0000	NaN	NaN	NaN	CWE-59	

	vulnerable_examples	benign_examples	is_unseen_cwe
8	0	3	0
10	0	3	0
1	6	950	0
14	22	177	0
2	21	722	0
4	27	692	0
0	15	556	0
5	11	656	0
12	2	636	0
7	1	112	0
9	1	50	0
6	1	33	0
3	2	25	0
13	1	1	0
11	1	0	0

Best CWEs (test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
12	638	0.0031	1.0000	1.0000	1.0000	1.0000	2	0	636	
7	113	0.0088	1.0000	1.0000	1.0000	1.0000	1	0	112	
9	51	0.0196	1.0000	1.0000	1.0000	1.0000	1	0	50	
6	34	0.0294	1.0000	1.0000	1.0000	1.0000	1	0	33	
3	27	0.0741	1.0000	1.0000	1.0000	1.0000	2	0	25	
13	2	0.5000	1.0000	1.0000	1.0000	1.0000	1	0	1	
11	1	1.0000	1.0000	1.0000	1.0000	1.0000	1	0	0	
4	719	0.0376	0.9958	0.9615	0.9259	0.9434	25	1	691	

0	571	0.0263	0.9965	0.9333	0.9333	0.9333	14	1	555
5	667	0.0165	0.9955	0.7857	1.0000	0.8800	11	3	653
2	743	0.0283	0.9933	0.9444	0.8095	0.8718	17	1	721
14	199	0.1106	0.9497	0.8333	0.6818	0.7500	15	3	174
1	956	0.0063	0.9969	0.8000	0.6667	0.7273	4	1	949
8	3	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	3
10	3	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	3

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
12	0	0.0000	0.0000	NaN	NaN	NaN	CWE-617	
7	0	0.0000	0.0000	NaN	NaN	NaN	CWE-369	
9	0	0.0000	0.0000	NaN	NaN	NaN	CWE-400	
6	0	0.0000	0.0000	NaN	NaN	NaN	CWE-362	
3	0	0.0000	0.0000	NaN	NaN	NaN	CWE-190	
13	0	0.0000	0.0000	NaN	NaN	NaN	CWE-665	
11	0	NaN	0.0000	NaN	NaN	NaN	CWE-59	
4	2	0.0014	0.0741	NaN	NaN	NaN	CWE-20	
0	1	0.0018	0.0667	NaN	NaN	NaN	CWE-119	
5	0	0.0046	0.0000	NaN	NaN	NaN	CWE-269	
2	4	0.0014	0.1905	NaN	NaN	NaN	CWE-189	
14	7	0.0169	0.3182	NaN	NaN	NaN	CWE-787	
1	2	0.0011	0.3333	NaN	NaN	NaN	CWE-125	
8	0	0.0000	NaN	NaN	NaN	NaN	CWE-399	
10	0	0.0000	NaN	NaN	NaN	NaN	CWE-476	

	vulnerable_examples	benign_examples	is_unseen_cwe
12	2	636	0
7	1	112	0
9	1	50	0
6	1	33	0
3	2	25	0
13	1	1	0
11	1	0	0
4	27	692	0
0	15	556	0
5	11	656	0
2	21	722	0
14	22	177	0
1	6	950	0
8	0	3	0
10	0	3	0

Seen CWE table (test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
8	3	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	3	
10	3	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	3	
1	956	0.0063	0.9969	0.8000	0.6667	0.7273	4	1	949	

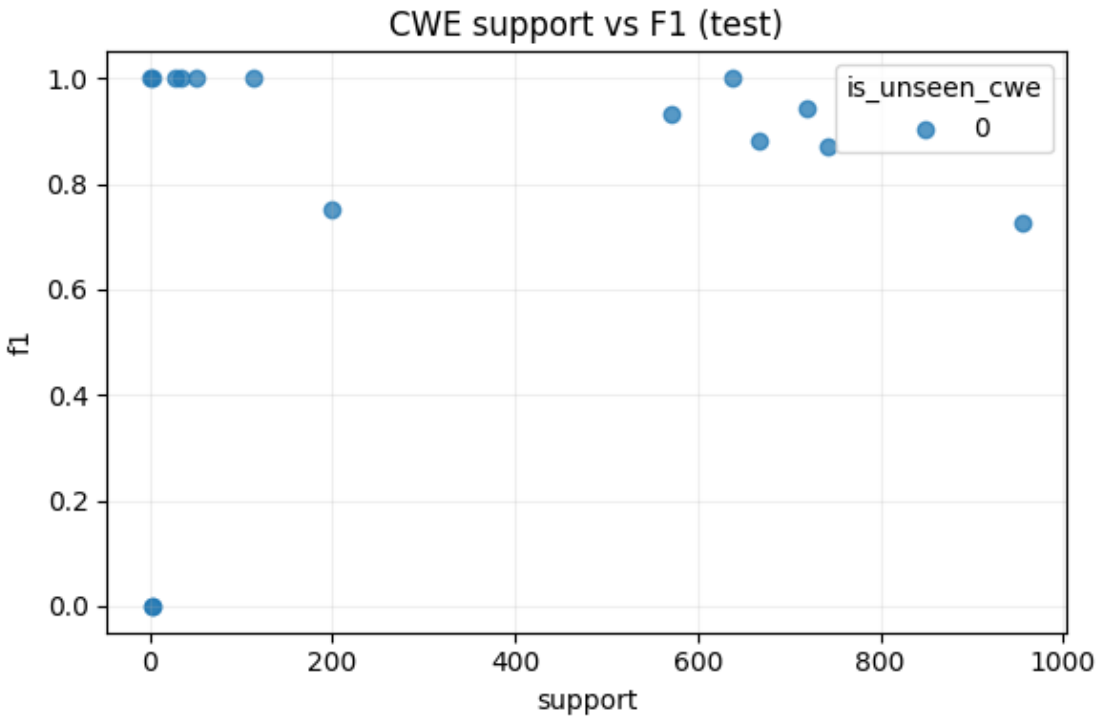
14	199	0.1106	0.9497	0.8333	0.6818	0.7500	15	3	174
2	743	0.0283	0.9933	0.9444	0.8095	0.8718	17	1	721
5	667	0.0165	0.9955	0.7857	1.0000	0.8800	11	3	653
0	571	0.0263	0.9965	0.9333	0.9333	0.9333	14	1	555
4	719	0.0376	0.9958	0.9615	0.9259	0.9434	25	1	691
12	638	0.0031	1.0000	1.0000	1.0000	1.0000	2	0	636
7	113	0.0088	1.0000	1.0000	1.0000	1.0000	1	0	112
9	51	0.0196	1.0000	1.0000	1.0000	1.0000	1	0	50
6	34	0.0294	1.0000	1.0000	1.0000	1.0000	1	0	33
3	27	0.0741	1.0000	1.0000	1.0000	1.0000	2	0	25
13	2	0.5000	1.0000	1.0000	1.0000	1.0000	1	0	1
11	1	1.0000	1.0000	1.0000	1.0000	1.0000	1	0	0

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
8	0	0.0000	NaN	NaN	NaN	NaN	CWE-399	
10	0	0.0000	NaN	NaN	NaN	NaN	CWE-476	
1	2	0.0011	0.3333	NaN	NaN	NaN	CWE-125	
14	7	0.0169	0.3182	NaN	NaN	NaN	CWE-787	
2	4	0.0014	0.1905	NaN	NaN	NaN	CWE-189	
5	0	0.0046	0.0000	NaN	NaN	NaN	CWE-269	
0	1	0.0018	0.0667	NaN	NaN	NaN	CWE-119	
4	2	0.0014	0.0741	NaN	NaN	NaN	CWE-20	
12	0	0.0000	0.0000	NaN	NaN	NaN	CWE-617	
7	0	0.0000	0.0000	NaN	NaN	NaN	CWE-369	
9	0	0.0000	0.0000	NaN	NaN	NaN	CWE-400	
6	0	0.0000	0.0000	NaN	NaN	NaN	CWE-362	
3	0	0.0000	0.0000	NaN	NaN	NaN	CWE-190	
13	0	0.0000	0.0000	NaN	NaN	NaN	CWE-665	
11	0	NaN	0.0000	NaN	NaN	NaN	CWE-59	

	vulnerable_examples	benign_examples	is_unseen_cwe
8	0	3	0
10	0	3	0
1	6	950	0
14	22	177	0
2	21	722	0
5	11	656	0
0	15	556	0
4	27	692	0
12	2	636	0
7	1	112	0
9	1	50	0
6	1	33	0
3	2	25	0
13	1	1	0
11	1	0	0

Unseen CWE table (test)

```
Empty DataFrame
Columns: [support, positive_rate, accuracy, precision, recall, f1, TP, FP, TN,␣
 ↪FN, fp_rate, fn_rate, roc_auc, pr_auc, brier_score, cwe_id,␣
 ↪vulnerable_examples, benign_examples, is_unseen_cwe]
Index: []
```



Caution: 4 CWE rows have support < 10 in test.

1.7.3 Per-CWE: unseen_val

Worst CWEs by F1 (unseen_val)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	\
34	50	0.0400	0.9600	0.0000	0.0000	0.0000	0	0	
39	37	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	
23	20	0.0500	0.9500	0.0000	0.0000	0.0000	0	0	
1	5	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	
28	2	0.0000	0.5000	0.0000	0.0000	0.0000	0	1	
27	1	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	
25	153	0.0065	0.5686	0.0149	1.0000	0.0294	1	66	
29	209	0.0287	0.0861	0.0256	0.8333	0.0498	5	190	
4	24	0.0417	0.5000	0.0769	1.0000	0.1429	1	12	
38	49	0.0204	0.7959	0.0909	1.0000	0.1667	1	10	

12	1757	0.0393	0.8014	0.1535	0.8986	0.2622	62	342	
33	436	0.0344	0.9335	0.2812	0.6000	0.3830	9	23	
9	1373	0.0757	0.8339	0.2905	0.8269	0.4300	86	210	
5	581	0.0740	0.8451	0.3178	0.9535	0.4767	41	88	
19	132	0.0152	0.9848	0.5000	0.5000	0.5000	1	1	

	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
34	48	2	0.0000	1.0000	NaN	NaN	NaN	CWE-78	
39	37	0	0.0000	NaN	NaN	NaN	NaN	CWE-93	
23	19	1	0.0000	1.0000	NaN	NaN	NaN	CWE-502	
1	5	0	0.0000	NaN	NaN	NaN	NaN	CWE-120	
28	1	0	0.5000	NaN	NaN	NaN	NaN	CWE-682	
27	1	0	0.0000	NaN	NaN	NaN	NaN	CWE-665	
25	86	0	0.4342	0.0000	NaN	NaN	NaN	CWE-611	
29	13	1	0.9360	0.1667	NaN	NaN	NaN	CWE-704	
4	11	0	0.5217	0.0000	NaN	NaN	NaN	CWE-172	
38	38	0	0.2083	0.0000	NaN	NaN	NaN	CWE-909	
12	1346	7	0.2026	0.1014	NaN	NaN	NaN	CWE-264	
33	398	6	0.0546	0.4000	NaN	NaN	NaN	CWE-772	
9	1059	18	0.1655	0.1731	NaN	NaN	NaN	CWE-200	
5	450	2	0.1636	0.0465	NaN	NaN	NaN	CWE-189	
19	129	1	0.0077	0.5000	NaN	NaN	NaN	CWE-400	

	vulnerable_examples	benign_examples	is_unseen_cwe
34	2	48	1
39	0	37	1
23	1	19	1
1	0	5	1
28	0	2	1
27	0	1	1
25	1	152	1
29	6	203	1
4	1	23	1
38	1	48	1
12	69	1688	1
33	15	421	1
9	104	1269	1
5	43	538	1
19	2	130	1

Worst CWEs by recall (unseen_val)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
34	50	0.0400	0.9600	0.0000	0.0000	0.0000	0	0	48	
39	37	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	37	
23	20	0.0500	0.9500	0.0000	0.0000	0.0000	0	0	19	
1	5	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	5	
28	2	0.0000	0.5000	0.0000	0.0000	0.0000	0	1	1	

27	1	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	1
31	259	0.0232	0.9807	0.6000	0.5000	0.5455	3	2	251
19	132	0.0152	0.9848	0.5000	0.5000	0.5000	1	1	129
36	57	0.0351	0.9825	1.0000	0.5000	0.6667	1	0	55
6	8	0.5000	0.5000	0.5000	0.5000	0.5000	2	2	2
18	521	0.0480	0.9731	0.8235	0.5600	0.6667	14	3	493
33	436	0.0344	0.9335	0.2812	0.6000	0.3830	9	23	398
21	414	0.0435	0.9638	0.5789	0.6111	0.5946	11	8	388
37	369	0.0163	0.9946	1.0000	0.6667	0.8000	4	0	363
14	60	0.0500	0.9833	1.0000	0.6667	0.8000	2	0	57

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
34	2	0.0000	1.0000	NaN	NaN	NaN	CWE-78	
39	0	0.0000	NaN	NaN	NaN	NaN	CWE-93	
23	1	0.0000	1.0000	NaN	NaN	NaN	CWE-502	
1	0	0.0000	NaN	NaN	NaN	NaN	CWE-120	
28	0	0.5000	NaN	NaN	NaN	NaN	CWE-682	
27	0	0.0000	NaN	NaN	NaN	NaN	CWE-665	
31	3	0.0079	0.5000	NaN	NaN	NaN	CWE-754	
19	1	0.0077	0.5000	NaN	NaN	NaN	CWE-400	
36	1	0.0000	0.5000	NaN	NaN	NaN	CWE-834	
6	2	0.5000	0.5000	NaN	NaN	NaN	CWE-19	
18	11	0.0060	0.4400	NaN	NaN	NaN	CWE-399	
33	6	0.0546	0.4000	NaN	NaN	NaN	CWE-772	
21	7	0.0202	0.3889	NaN	NaN	NaN	CWE-416	
37	2	0.0000	0.3333	NaN	NaN	NaN	CWE-835	
14	1	0.0000	0.3333	NaN	NaN	NaN	CWE-310	

	vulnerable_examples	benign_examples	is_unseen_cwe
34	2	48	1
39	0	37	1
23	1	19	1
1	0	5	1
28	0	2	1
27	0	1	1
31	6	253	1
19	2	130	1
36	2	55	1
6	4	4	1
18	25	496	1
33	15	421	1
21	18	396	1
37	6	363	1
14	3	57	1

Best CWEs (unseen_val)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
40	118	0.0169	1.0000	1.0000	1.0000	1.0000	2	0	116	
26	104	0.0096	1.0000	1.0000	1.0000	1.0000	1	0	103	
17	47	0.1277	1.0000	1.0000	1.0000	1.0000	6	0	41	
24	40	0.0750	1.0000	1.0000	1.0000	1.0000	3	0	37	
30	39	0.0256	1.0000	1.0000	1.0000	1.0000	1	0	38	
15	1	1.0000	1.0000	1.0000	1.0000	1.0000	1	0	0	
20	107	0.0374	0.9907	1.0000	0.7500	0.8571	3	0	103	
37	369	0.0163	0.9946	1.0000	0.6667	0.8000	4	0	363	
14	60	0.0500	0.9833	1.0000	0.6667	0.8000	2	0	57	
0	2677	0.0527	0.9787	0.8750	0.6950	0.7747	98	14	2522	
7	741	0.1066	0.9406	0.6522	0.9494	0.7732	75	40	622	
11	382	0.0602	0.9581	0.5946	0.9565	0.7333	22	15	344	
18	521	0.0480	0.9731	0.8235	0.5600	0.6667	14	3	493	
10	119	0.0336	0.9748	0.6000	0.7500	0.6667	3	2	113	
16	107	0.0187	0.9813	0.5000	1.0000	0.6667	2	2	103	

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
40	0	0.0000	0.0000	NaN	NaN	NaN	CWE-94	
26	0	0.0000	0.0000	NaN	NaN	NaN	CWE-617	
17	0	0.0000	0.0000	NaN	NaN	NaN	CWE-369	
24	0	0.0000	0.0000	NaN	NaN	NaN	CWE-59	
30	0	0.0000	0.0000	NaN	NaN	NaN	CWE-732	
15	0	NaN	0.0000	NaN	NaN	NaN	CWE-345	
20	1	0.0000	0.2500	NaN	NaN	NaN	CWE-415	
37	2	0.0000	0.3333	NaN	NaN	NaN	CWE-835	
14	1	0.0000	0.3333	NaN	NaN	NaN	CWE-310	
0	43	0.0055	0.3050	NaN	NaN	NaN	CWE-119	
7	4	0.0604	0.0506	NaN	NaN	NaN	CWE-190	
11	1	0.0418	0.0435	NaN	NaN	NaN	CWE-254	
18	11	0.0060	0.4400	NaN	NaN	NaN	CWE-399	
10	1	0.0174	0.2500	NaN	NaN	NaN	CWE-22	
16	0	0.0190	0.0000	NaN	NaN	NaN	CWE-362	

	vulnerable_examples	benign_examples	is_unseen_cwe
40	2	116	1
26	1	103	1
17	6	41	1
24	3	37	1
30	1	38	1
15	1	0	1
20	4	103	1
37	6	363	1
14	3	57	1
0	141	2536	1
7	79	662	1
11	23	359	1
18	25	496	1

```
10                    4               115               1
16                    2               105               1
```

Seen CWE table (unseen_val)

```
Empty DataFrame
Columns: [support, positive_rate, accuracy, precision, recall, f1, TP, FP, TN,␣
 ↪FN, fp_rate, fn_rate, roc_auc, pr_auc, brier_score, cwe_id,␣
 ↪vulnerable_examples, benign_examples, is_unseen_cwe]
Index: []
```

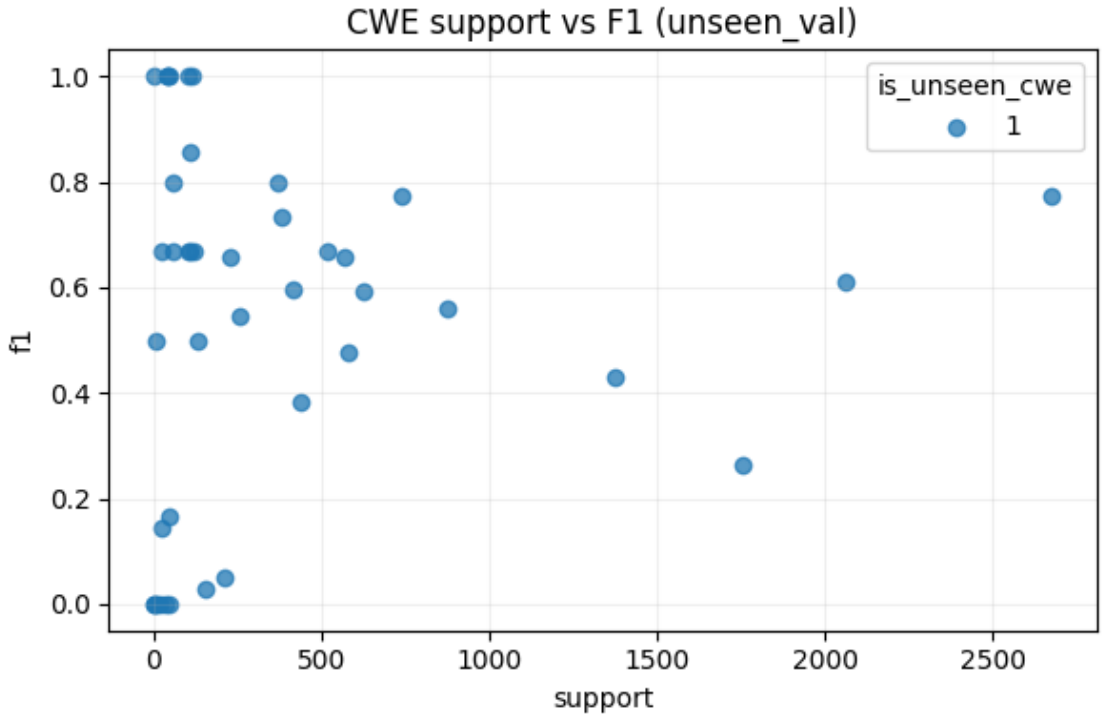
Unseen CWE table (unseen_val)

```
    support  positive_rate  accuracy  precision  recall      f1  TP   FP  \
34       50          0.0400    0.9600     0.0000  0.0000  0.0000   0    0
39       37          0.0000    1.0000     0.0000  0.0000  0.0000   0    0
23       20          0.0500    0.9500     0.0000  0.0000  0.0000   0    0
1         5          0.0000    1.0000     0.0000  0.0000  0.0000   0    0
28        2          0.0000    0.5000     0.0000  0.0000  0.0000   0    1
27        1          0.0000    1.0000     0.0000  0.0000  0.0000   0    0
25      153          0.0065    0.5686     0.0149  1.0000  0.0294   1   66
29      209          0.0287    0.0861     0.0256  0.8333  0.0498   5  190
4        24          0.0417    0.5000     0.0769  1.0000  0.1429   1   12
38       49          0.0204    0.7959     0.0909  1.0000  0.1667   1   10
12     1757          0.0393    0.8014     0.1535  0.8986  0.2622  62  342
33      436          0.0344    0.9335     0.2812  0.6000  0.3830   9   23
9      1373          0.0757    0.8339     0.2905  0.8269  0.4300  86  210
5       581          0.0740    0.8451     0.3178  0.9535  0.4767  41   88
19      132          0.0152    0.9848     0.5000  0.5000  0.5000   1    1

      TN  FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score   cwe_id  \
34    48   2   0.0000   1.0000      NaN     NaN          NaN   CWE-78
39    37   0   0.0000      NaN      NaN     NaN          NaN   CWE-93
23    19   1   0.0000   1.0000      NaN     NaN          NaN  CWE-502
1      5   0   0.0000      NaN      NaN     NaN          NaN  CWE-120
28     1   0   0.5000      NaN      NaN     NaN          NaN  CWE-682
27     1   0   0.0000      NaN      NaN     NaN          NaN  CWE-665
25    86   0   0.4342   0.0000      NaN     NaN          NaN  CWE-611
29    13   1   0.9360   0.1667      NaN     NaN          NaN  CWE-704
4     11   0   0.5217   0.0000      NaN     NaN          NaN  CWE-172
38    38   0   0.2083   0.0000      NaN     NaN          NaN  CWE-909
12  1346   7   0.2026   0.1014      NaN     NaN          NaN  CWE-264
33   398   6   0.0546   0.4000      NaN     NaN          NaN  CWE-772
9   1059  18   0.1655   0.1731      NaN     NaN          NaN  CWE-200
5    450   2   0.1636   0.0465      NaN     NaN          NaN  CWE-189
19   129   1   0.0077   0.5000      NaN     NaN          NaN  CWE-400

    vulnerable_examples  benign_examples  is_unseen_cwe
```

34	2	48	1
39	0	37	1
23	1	19	1
1	0	5	1
28	0	2	1
27	0	1	1
25	1	152	1
29	6	203	1
4	1	23	1
38	1	48	1
12	69	1688	1
33	15	421	1
9	104	1269	1
5	43	538	1
19	2	130	1



Caution: 5 CWE rows have support < 10 in unseen_val.

1.7.4 Per-CWE: unseen_test

Worst CWEs by F1 (unseen_test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
11	64	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	64	
17	51	0.0000	0.8824	0.0000	0.0000	0.0000	0	6	45	

44	42	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	42
26	20	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	20
15	16	0.0625	0.9375	0.0000	0.0000	0.0000	0	0	15
33	16	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	16
49	15	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	15
12	13	0.0769	0.9231	0.0000	0.0000	0.0000	0	0	12
4	10	0.1000	0.9000	0.0000	0.0000	0.0000	0	0	9
37	5	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	5
21	2	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	2
25	227	0.0485	0.9648	1.0000	0.2727	0.4286	3	0	216
46	253	0.0277	0.9842	1.0000	0.4286	0.6000	3	0	246
23	23	0.3043	0.8261	1.0000	0.4286	0.6000	3	0	16
36	168	0.0357	0.9702	0.5556	0.8333	0.6667	5	4	158

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
11	0	0.0000	NaN	NaN	NaN	NaN	CWE-254	
17	0	0.1176	NaN	NaN	NaN	NaN	CWE-290	
44	0	0.0000	NaN	NaN	NaN	NaN	CWE-79	
26	0	0.0000	NaN	NaN	NaN	NaN	CWE-388	
15	1	0.0000	1.0000	NaN	NaN	NaN	CWE-284	
33	0	0.0000	NaN	NaN	NaN	NaN	CWE-502	
49	0	0.0000	NaN	NaN	NaN	NaN	CWE-90	
12	1	0.0000	1.0000	NaN	NaN	NaN	CWE-255	
4	1	0.0000	1.0000	NaN	NaN	NaN	CWE-17	
37	0	0.0000	NaN	NaN	NaN	NaN	CWE-665	
21	0	0.0000	NaN	NaN	NaN	NaN	CWE-327	
25	8	0.0000	0.7273	NaN	NaN	NaN	CWE-369	
46	4	0.0000	0.5714	NaN	NaN	NaN	CWE-834	
23	4	0.0000	0.5714	NaN	NaN	NaN	CWE-354	
36	1	0.0247	0.1667	NaN	NaN	NaN	CWE-617	

	vulnerable_examples	benign_examples	is_unseen_cwe
11	0	64	1
17	0	51	1
44	0	42	1
26	0	20	1
15	1	15	1
33	0	16	1
49	0	15	1
12	1	12	1
4	1	9	1
37	0	5	1
21	0	2	1
25	11	216	1
46	7	246	1
23	7	16	1
36	6	162	1

Worst CWEs by recall (unseen_test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
11	64	0.0000	1.0000	0.0	0.0000	0.0000	0	0	64	
17	51	0.0000	0.8824	0.0	0.0000	0.0000	0	6	45	
44	42	0.0000	1.0000	0.0	0.0000	0.0000	0	0	42	
26	20	0.0000	1.0000	0.0	0.0000	0.0000	0	0	20	
15	16	0.0625	0.9375	0.0	0.0000	0.0000	0	0	15	
33	16	0.0000	1.0000	0.0	0.0000	0.0000	0	0	16	
49	15	0.0000	1.0000	0.0	0.0000	0.0000	0	0	15	
12	13	0.0769	0.9231	0.0	0.0000	0.0000	0	0	12	
4	10	0.1000	0.9000	0.0	0.0000	0.0000	0	0	9	
37	5	0.0000	1.0000	0.0	0.0000	0.0000	0	0	5	
21	2	0.0000	1.0000	0.0	0.0000	0.0000	0	0	2	
25	227	0.0485	0.9648	1.0	0.2727	0.4286	3	0	216	
46	253	0.0277	0.9842	1.0	0.4286	0.6000	3	0	246	
23	23	0.3043	0.8261	1.0	0.4286	0.6000	3	0	16	
31	27	0.0741	0.9630	1.0	0.5000	0.6667	1	0	25	

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
11	0	0.0000	NaN	NaN	NaN	NaN	CWE-254	
17	0	0.1176	NaN	NaN	NaN	NaN	CWE-290	
44	0	0.0000	NaN	NaN	NaN	NaN	CWE-79	
26	0	0.0000	NaN	NaN	NaN	NaN	CWE-388	
15	1	0.0000	1.0000	NaN	NaN	NaN	CWE-284	
33	0	0.0000	NaN	NaN	NaN	NaN	CWE-502	
49	0	0.0000	NaN	NaN	NaN	NaN	CWE-90	
12	1	0.0000	1.0000	NaN	NaN	NaN	CWE-255	
4	1	0.0000	1.0000	NaN	NaN	NaN	CWE-17	
37	0	0.0000	NaN	NaN	NaN	NaN	CWE-665	
21	0	0.0000	NaN	NaN	NaN	NaN	CWE-327	
25	8	0.0000	0.7273	NaN	NaN	NaN	CWE-369	
46	4	0.0000	0.5714	NaN	NaN	NaN	CWE-834	
23	4	0.0000	0.5714	NaN	NaN	NaN	CWE-354	
31	1	0.0000	0.5000	NaN	NaN	NaN	CWE-426	

	vulnerable_examples	benign_examples	is_unseen_cwe
11	0	64	1
17	0	51	1
44	0	42	1
26	0	20	1
15	1	15	1
33	0	16	1
49	0	15	1
12	1	12	1
4	1	9	1
37	0	5	1
21	0	2	1

25	11	216	1
46	7	246	1
23	7	16	1
31	2	25	1

Best CWEs (unseen_test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	FN	\
39	331	0.0151	1.0	1.0	1.0	1.0	5	0	326	0	
35	279	0.0466	1.0	1.0	1.0	1.0	13	0	266	0	
42	193	0.0311	1.0	1.0	1.0	1.0	6	0	187	0	
10	77	0.0130	1.0	1.0	1.0	1.0	1	0	76	0	
34	74	0.0135	1.0	1.0	1.0	1.0	1	0	73	0	
14	59	0.0169	1.0	1.0	1.0	1.0	1	0	58	0	
3	44	0.0227	1.0	1.0	1.0	1.0	1	0	43	0	
38	41	0.1220	1.0	1.0	1.0	1.0	5	0	36	0	
20	40	0.1000	1.0	1.0	1.0	1.0	4	0	36	0	
24	39	0.0256	1.0	1.0	1.0	1.0	1	0	38	0	
48	24	0.1250	1.0	1.0	1.0	1.0	3	0	21	0	
50	18	0.0556	1.0	1.0	1.0	1.0	1	0	17	0	
40	10	0.2000	1.0	1.0	1.0	1.0	2	0	8	0	
45	2	0.5000	1.0	1.0	1.0	1.0	1	0	1	0	
2	1	1.0000	1.0	1.0	1.0	1.0	1	0	0	0	

	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
39	0.0	0.0	NaN	NaN	NaN	CWE-704	
35	0.0	0.0	NaN	NaN	NaN	CWE-59	
42	0.0	0.0	NaN	NaN	NaN	CWE-78	
10	0.0	0.0	NaN	NaN	NaN	CWE-22	
34	0.0	0.0	NaN	NaN	NaN	CWE-532	
14	0.0	0.0	NaN	NaN	NaN	CWE-269	
3	0.0	0.0	NaN	NaN	NaN	CWE-134	
38	0.0	0.0	NaN	NaN	NaN	CWE-674	
20	0.0	0.0	NaN	NaN	NaN	CWE-320	
24	0.0	0.0	NaN	NaN	NaN	CWE-362	
48	0.0	0.0	NaN	NaN	NaN	CWE-89	
50	0.0	0.0	NaN	NaN	NaN	CWE-918	
40	0.0	0.0	NaN	NaN	NaN	CWE-74	
45	0.0	0.0	NaN	NaN	NaN	CWE-824	
2	NaN	0.0	NaN	NaN	NaN	CWE-129	

	vulnerable_examples	benign_examples	is_unseen_cwe
39	5	326	1
35	13	266	1
42	6	187	1
10	1	76	1
34	1	73	1
14	1	58	1

3	1	43	1
38	5	36	1
20	4	36	1
24	1	38	1
48	3	21	1
50	1	17	1
40	2	8	1
45	1	1	1
2	1	0	1

Seen CWE table (unseen_test)

```
Empty DataFrame
Columns: [support, positive_rate, accuracy, precision, recall, f1, TP, FP, TN,␣
 ↪FN, fp_rate, fn_rate, roc_auc, pr_auc, brier_score, cwe_id,␣
 ↪vulnerable_examples, benign_examples, is_unseen_cwe]
Index: []
```

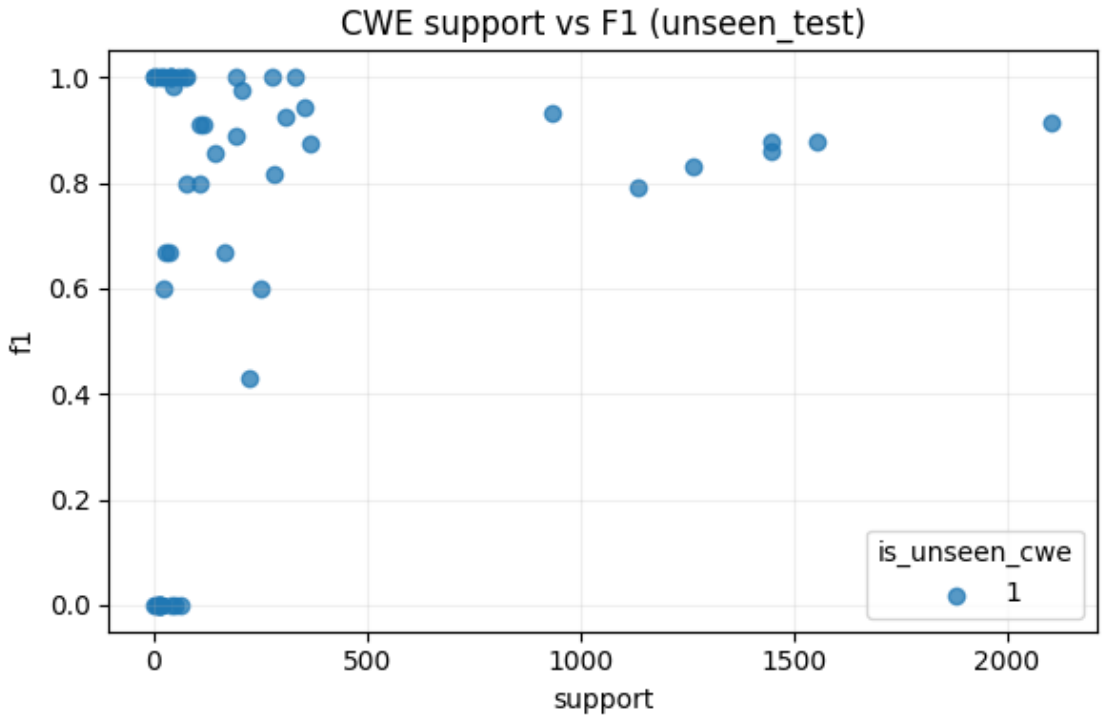
Unseen CWE table (unseen_test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
11	64	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	64	
17	51	0.0000	0.8824	0.0000	0.0000	0.0000	0	6	45	
44	42	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	42	
26	20	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	20	
15	16	0.0625	0.9375	0.0000	0.0000	0.0000	0	0	15	
33	16	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	16	
49	15	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	15	
12	13	0.0769	0.9231	0.0000	0.0000	0.0000	0	0	12	
4	10	0.1000	0.9000	0.0000	0.0000	0.0000	0	0	9	
37	5	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	5	
21	2	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	2	
25	227	0.0485	0.9648	1.0000	0.2727	0.4286	3	0	216	
46	253	0.0277	0.9842	1.0000	0.4286	0.6000	3	0	246	
23	23	0.3043	0.8261	1.0000	0.4286	0.6000	3	0	16	
36	168	0.0357	0.9702	0.5556	0.8333	0.6667	5	4	158	

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	cwe_id	\
11	0	0.0000	NaN	NaN	NaN	NaN	CWE-254	
17	0	0.1176	NaN	NaN	NaN	NaN	CWE-290	
44	0	0.0000	NaN	NaN	NaN	NaN	CWE-79	
26	0	0.0000	NaN	NaN	NaN	NaN	CWE-388	
15	1	0.0000	1.0000	NaN	NaN	NaN	CWE-284	
33	0	0.0000	NaN	NaN	NaN	NaN	CWE-502	
49	0	0.0000	NaN	NaN	NaN	NaN	CWE-90	
12	1	0.0000	1.0000	NaN	NaN	NaN	CWE-255	
4	1	0.0000	1.0000	NaN	NaN	NaN	CWE-17	
37	0	0.0000	NaN	NaN	NaN	NaN	CWE-665	

21	0	0.0000	NaN	NaN	NaN	NaN	CWE-327
25	8	0.0000	0.7273	NaN	NaN	NaN	CWE-369
46	4	0.0000	0.5714	NaN	NaN	NaN	CWE-834
23	4	0.0000	0.5714	NaN	NaN	NaN	CWE-354
36	1	0.0247	0.1667	NaN	NaN	NaN	CWE-617

	vulnerable_examples	benign_examples	is_unseen_cwe
11	0	64	1
17	0	51	1
44	0	42	1
26	0	20	1
15	1	15	1
33	0	16	1
49	0	15	1
12	1	12	1
4	1	9	1
37	0	5	1
21	0	2	1
25	11	216	1
46	7	246	1
23	7	16	1
36	6	162	1



Caution: 4 CWE rows have support < 10 in unseen_test.

1.8 Per-Project Analysis

This section checks whether errors are concentrated in a few repositories or whether the model underperforms consistently across projects.

[10]:
```
per_project_by_split = {}
for split_name in all_eval_split_names:
    split_df = frames[split_name]
    per_project = pea.per_project_metrics(split_df)
    if per_project.empty:
        print(f"TODO: `project` is missing in {split_name}; skipping␣
↪per-project analysis for this split.")
        continue
    per_project_by_split[split_name] = per_project
    pea.save_dataframe(per_project, OUTPUT_DIR /␣
↪f"per_project_metrics_{split_name}.csv")

    display(Markdown(f"### Per-Project: {split_name}"))
    display(Markdown(f"#### Largest projects ({split_name})"))
    display(per_project.sort_values("support", ascending=False).head(15).
↪round(4))
    display(Markdown(f"#### Worst projects by F1 ({split_name})"))
    display(per_project.sort_values(["f1", "support"], ascending=[True, False]).
↪head(15).round(4))
    display(Markdown(f"#### Highest FP count projects ({split_name})"))
    display(per_project.sort_values(["FP", "support"], ascending=[False,␣
↪False]).head(15).round(4))
    display(Markdown(f"#### Highest FN count projects ({split_name})"))
    display(per_project.sort_values(["FN", "support"], ascending=[False,␣
↪False]).head(15).round(4))

    fig = pea.plot_support_scatter(per_project, "support", "f1", f"Project␣
↪support vs F1 ({split_name})")
    display(fig)
    pea.save_figure(fig, OUTPUT_DIR / f"per_project_support_vs_f1_{split_name}.
↪png")
    plt.close(fig)

# Cross-split project summaries (on merged eval_df)
if "is_unseen_cwe" in eval_df.columns and "project" in eval_df.columns:
    project_seen_crosstab = eval_df.pivot_table(index="project",␣
↪columns="is_unseen_cwe", values="y_true", aggfunc="size", fill_value=0)
    display(Markdown("### Project x seen/unseen counts (merged eval splits)"))
    display(project_seen_crosstab.head(25))
if "cwe_id" in eval_df.columns and "project" in eval_df.columns:
    project_cwe_counts = eval_df.groupby("project")["cwe_id"].nunique().
↪sort_values(ascending=False).rename("unique_cwes").reset_index()
```

```
    display(Markdown("### Project x CWE count (merged eval splits)"))
    display(project_cwe_counts.head(25))
```

1.8.1 Per-Project: val

Largest projects (val)

```
   support  positive_rate  accuracy  precision  recall     f1   TP  FP    TN  \
0     8465         0.0465    0.9894     0.9578  0.8071  0.876  318  14  8057

   FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score project  \
0  76   0.0017   0.1929      NaN     NaN          NaN   linux

   vulnerable_examples  benign_examples  unique_cwes
0                  394             8071           36
```

Worst projects by F1 (val)

```
   support  positive_rate  accuracy  precision  recall     f1   TP  FP    TN  \
0     8465         0.0465    0.9894     0.9578  0.8071  0.876  318  14  8057

   FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score project  \
0  76   0.0017   0.1929      NaN     NaN          NaN   linux

   vulnerable_examples  benign_examples  unique_cwes
0                  394             8071           36
```

Highest FP count projects (val)

```
   support  positive_rate  accuracy  precision  recall     f1   TP  FP    TN  \
0     8465         0.0465    0.9894     0.9578  0.8071  0.876  318  14  8057

   FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score project  \
0  76   0.0017   0.1929      NaN     NaN          NaN   linux

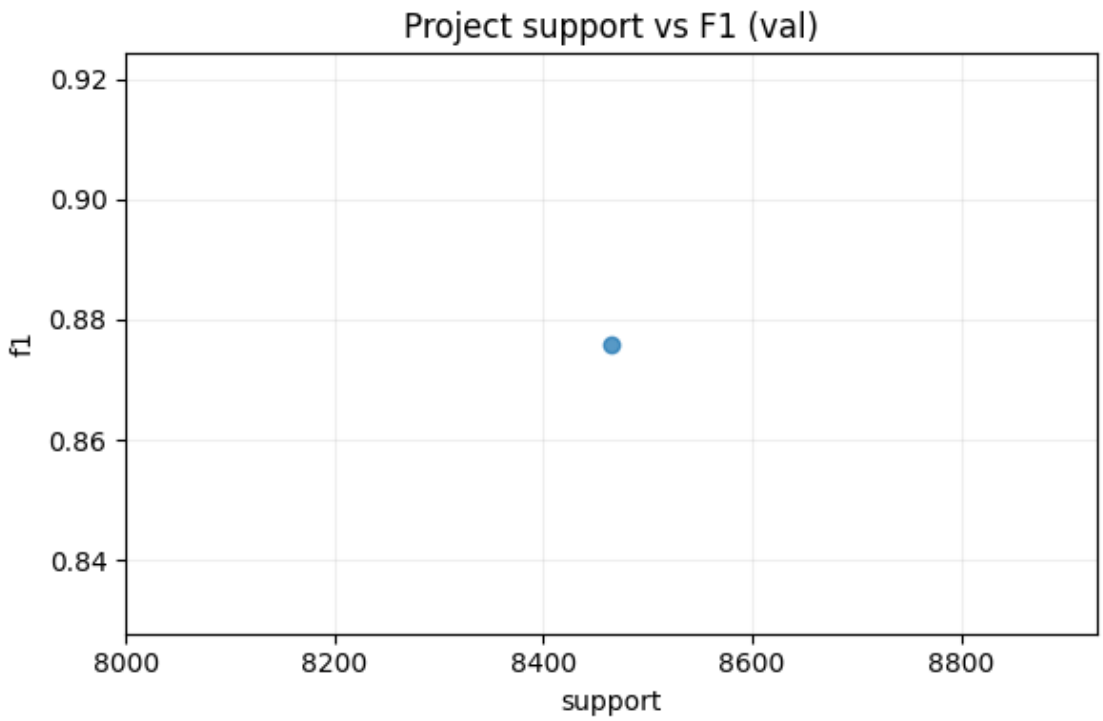
   vulnerable_examples  benign_examples  unique_cwes
0                  394             8071           36
```

Highest FN count projects (val)

```
   support  positive_rate  accuracy  precision  recall     f1   TP  FP    TN  \
0     8465         0.0465    0.9894     0.9578  0.8071  0.876  318  14  8057

   FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score project  \
0  76   0.0017   0.1929      NaN     NaN          NaN   linux

   vulnerable_examples  benign_examples  unique_cwes
0                  394             8071           36
```



1.8.2 Per-Project: test

Largest projects (test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
1	1204	0.0108	0.9975	0.9167	0.8462	0.8800	11	1	1190	
7	1087	0.0276	0.9991	0.9677	1.0000	0.9836	30	1	1056	
5	968	0.0341	0.9907	0.9286	0.7879	0.8525	26	2	933	
0	660	0.0167	0.9955	0.7857	1.0000	0.8800	11	3	646	
4	638	0.0031	1.0000	1.0000	1.0000	1.0000	2	0	636	
6	142	0.1268	0.9296	0.7857	0.6111	0.6875	11	3	121	
3	24	0.1250	1.0000	1.0000	1.0000	1.0000	3	0	21	
2	4	0.2500	1.0000	1.0000	1.0000	1.0000	1	0	3	

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project	\
1	2	0.0008	0.1538	NaN	NaN	NaN	gpac	
7	0	0.0009	0.0000	NaN	NaN	NaN	xserver	
5	7	0.0021	0.2121	NaN	NaN	NaN	poppler	
0	0	0.0046	0.0000	NaN	NaN	NaN	OpenJK	
4	0	0.0000	0.0000	NaN	NaN	NaN	ovs	
6	7	0.0242	0.3889	NaN	NaN	NaN	u-boot	
3	0	0.0000	0.0000	NaN	NaN	NaN	libXrandr	
2	0	0.0000	0.0000	NaN	NaN	NaN	libXpm	

	vulnerable_examples	benign_examples	unique_cwes
1	13	1191	4
7	30	1057	9
5	33	935	6
0	11	649	1
4	2	636	1
6	18	124	1
3	3	21	1
2	1	3	1

Worst projects by F1 (test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
6	142	0.1268	0.9296	0.7857	0.6111	0.6875	11	3	121	
5	968	0.0341	0.9907	0.9286	0.7879	0.8525	26	2	933	
1	1204	0.0108	0.9975	0.9167	0.8462	0.8800	11	1	1190	
0	660	0.0167	0.9955	0.7857	1.0000	0.8800	11	3	646	
7	1087	0.0276	0.9991	0.9677	1.0000	0.9836	30	1	1056	
4	638	0.0031	1.0000	1.0000	1.0000	1.0000	2	0	636	
3	24	0.1250	1.0000	1.0000	1.0000	1.0000	3	0	21	
2	4	0.2500	1.0000	1.0000	1.0000	1.0000	1	0	3	

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project	\
6	7	0.0242	0.3889	NaN	NaN	NaN	u-boot	
5	7	0.0021	0.2121	NaN	NaN	NaN	poppler	
1	2	0.0008	0.1538	NaN	NaN	NaN	gpac	
0	0	0.0046	0.0000	NaN	NaN	NaN	OpenJK	
7	0	0.0009	0.0000	NaN	NaN	NaN	xserver	
4	0	0.0000	0.0000	NaN	NaN	NaN	ovs	
3	0	0.0000	0.0000	NaN	NaN	NaN	libXrandr	
2	0	0.0000	0.0000	NaN	NaN	NaN	libXpm	

	vulnerable_examples	benign_examples	unique_cwes
6	18	124	1
5	33	935	6
1	13	1191	4
0	11	649	1
7	30	1057	9
4	2	636	1
3	3	21	1
2	1	3	1

Highest FP count projects (test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN	\
0	660	0.0167	0.9955	0.7857	1.0000	0.8800	11	3	646	
6	142	0.1268	0.9296	0.7857	0.6111	0.6875	11	3	121	
5	968	0.0341	0.9907	0.9286	0.7879	0.8525	26	2	933	

1	1204	0.0108	0.9975	0.9167	0.8462	0.8800	11	1	1190
7	1087	0.0276	0.9991	0.9677	1.0000	0.9836	30	1	1056
4	638	0.0031	1.0000	1.0000	1.0000	1.0000	2	0	636
3	24	0.1250	1.0000	1.0000	1.0000	1.0000	3	0	21
2	4	0.2500	1.0000	1.0000	1.0000	1.0000	1	0	3

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project
0	0	0.0046	0.0000	NaN	NaN	NaN	OpenJK
6	7	0.0242	0.3889	NaN	NaN	NaN	u-boot
5	7	0.0021	0.2121	NaN	NaN	NaN	poppler
1	2	0.0008	0.1538	NaN	NaN	NaN	gpac
7	0	0.0009	0.0000	NaN	NaN	NaN	xserver
4	0	0.0000	0.0000	NaN	NaN	NaN	ovs
3	0	0.0000	0.0000	NaN	NaN	NaN	libXrandr
2	0	0.0000	0.0000	NaN	NaN	NaN	libXpm

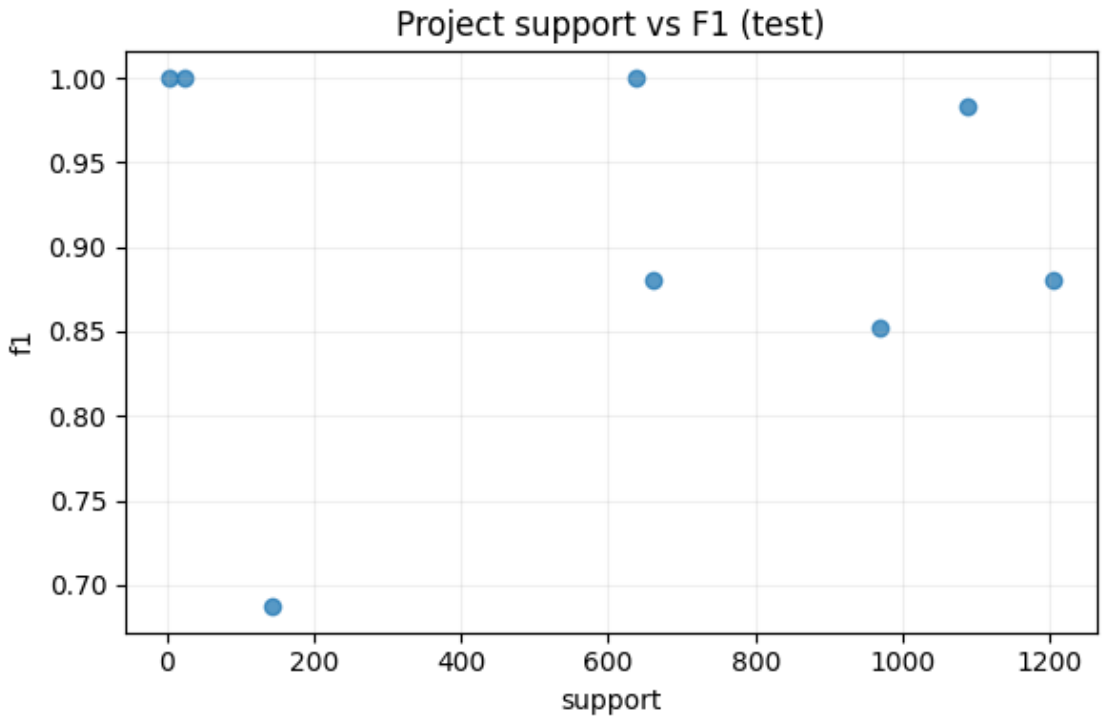
	vulnerable_examples	benign_examples	unique_cwes
0	11	649	1
6	18	124	1
5	33	935	6
1	13	1191	4
7	30	1057	9
4	2	636	1
3	3	21	1
2	1	3	1

Highest FN count projects (test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN
5	968	0.0341	0.9907	0.9286	0.7879	0.8525	26	2	933
6	142	0.1268	0.9296	0.7857	0.6111	0.6875	11	3	121
1	1204	0.0108	0.9975	0.9167	0.8462	0.8800	11	1	1190
7	1087	0.0276	0.9991	0.9677	1.0000	0.9836	30	1	1056
0	660	0.0167	0.9955	0.7857	1.0000	0.8800	11	3	646
4	638	0.0031	1.0000	1.0000	1.0000	1.0000	2	0	636
3	24	0.1250	1.0000	1.0000	1.0000	1.0000	3	0	21
2	4	0.2500	1.0000	1.0000	1.0000	1.0000	1	0	3

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project
5	7	0.0021	0.2121	NaN	NaN	NaN	poppler
6	7	0.0242	0.3889	NaN	NaN	NaN	u-boot
1	2	0.0008	0.1538	NaN	NaN	NaN	gpac
7	0	0.0009	0.0000	NaN	NaN	NaN	xserver
0	0	0.0046	0.0000	NaN	NaN	NaN	OpenJK
4	0	0.0000	0.0000	NaN	NaN	NaN	ovs
3	0	0.0000	0.0000	NaN	NaN	NaN	libXrandr
2	0	0.0000	0.0000	NaN	NaN	NaN	libXpm

	vulnerable_examples	benign_examples	unique_cwes
5	33	935	6
6	18	124	1
1	13	1191	4
7	30	1057	9
0	11	649	1
4	2	636	1
3	3	21	1
2	1	3	1



1.8.3 Per-Project: unseen_val

Largest projects (unseen_val)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	\
0	5287	0.0758	0.7314	0.2117	0.9327	0.3450	374	1393	
15	3042	0.0381	0.9915	0.9245	0.8448	0.8829	98	8	
1	2119	0.0835	0.9547	0.7872	0.6271	0.6981	111	30	
16	1965	0.0356	0.9878	0.8710	0.7714	0.8182	54	8	
13	1902	0.0526	0.9811	0.9000	0.7200	0.8000	72	8	
6	697	0.1363	0.9770	0.8692	0.9789	0.9208	93	14	
5	89	0.0449	0.9888	1.0000	0.7500	0.8571	3	0	
11	81	0.0123	1.0000	1.0000	1.0000	1.0000	1	0	
3	53	0.1132	0.9811	1.0000	0.8333	0.9091	5	0	
7	51	0.0588	0.9608	0.6000	1.0000	0.7500	3	2	

18	40	0.0500	0.9750	1.0000	0.5000	0.6667	1	0
2	36	0.0556	1.0000	1.0000	1.0000	1.0000	2	0
14	35	0.0857	0.9714	1.0000	0.6667	0.8000	2	0
8	30	0.0000	1.0000	0.0000	0.0000	0.0000	0	0
10	16	0.0625	0.9375	0.5000	1.0000	0.6667	1	1

	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project	\
0	3493	27	0.2851	0.0673	NaN	NaN	NaN	Android	
15	2918	18	0.0027	0.1552	NaN	NaN	NaN	qemu	
1	1912	66	0.0154	0.3729	NaN	NaN	NaN	ImageMagick	
16	1887	16	0.0042	0.2286	NaN	NaN	NaN	savannah	
13	1794	28	0.0044	0.2800	NaN	NaN	NaN	php	
6	588	2	0.0233	0.0211	NaN	NaN	NaN	jasper	
5	85	1	0.0000	0.2500	NaN	NaN	NaN	gstreamer	
11	80	0	0.0000	0.0000	NaN	NaN	NaN	openvpn	
3	47	1	0.0000	0.1667	NaN	NaN	NaN	abrt	
7	46	0	0.0417	0.0000	NaN	NaN	NaN	libvips	
18	38	1	0.0000	0.5000	NaN	NaN	NaN	zfs	
2	34	0	0.0000	0.0000	NaN	NaN	NaN	NetworkManager	
14	32	1	0.0000	0.3333	NaN	NaN	NaN	polkit	
8	30	0	0.0000	NaN	NaN	NaN	NaN	lynx-snapshots	
10	14	0	0.0667	0.0000	NaN	NaN	NaN	netdata	

	vulnerable_examples	benign_examples	unique_cwes
0	401	4886	22
15	116	2926	21
1	177	1942	21
16	70	1895	18
13	100	1802	16
6	95	602	8
5	4	85	4
11	1	80	2
3	6	47	2
7	3	48	2
18	2	38	1
2	2	34	2
14	3	32	1
8	0	30	1
10	1	15	1

Worst projects by F1 (unseen_val)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	\
8	30	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	
17	12	0.0833	0.9167	0.0000	0.0000	0.0000	0	0	
9	7	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	
0	5287	0.0758	0.7314	0.2117	0.9327	0.3450	374	1393	
18	40	0.0500	0.9750	1.0000	0.5000	0.6667	1	0	

10	16	0.0625	0.9375	0.5000	1.0000	0.6667	1	1
1	2119	0.0835	0.9547	0.7872	0.6271	0.6981	111	30
7	51	0.0588	0.9608	0.6000	1.0000	0.7500	3	2
13	1902	0.0526	0.9811	0.9000	0.7200	0.8000	72	8
14	35	0.0857	0.9714	1.0000	0.6667	0.8000	2	0
16	1965	0.0356	0.9878	0.8710	0.7714	0.8182	54	8
5	89	0.0449	0.9888	1.0000	0.7500	0.8571	3	0
15	3042	0.0381	0.9915	0.9245	0.8448	0.8829	98	8
3	53	0.1132	0.9811	1.0000	0.8333	0.9091	5	0
6	697	0.1363	0.9770	0.8692	0.9789	0.9208	93	14

	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project	\
8	30	0	0.0000	NaN	NaN	NaN	NaN	lynx-snapshots	
17	11	1	0.0000	1.0000	NaN	NaN	NaN	yubico-pam	
9	7	0	0.0000	NaN	NaN	NaN	NaN	mongoose-os	
0	3493	27	0.2851	0.0673	NaN	NaN	NaN	Android	
18	38	1	0.0000	0.5000	NaN	NaN	NaN	zfs	
10	14	0	0.0667	0.0000	NaN	NaN	NaN	netdata	
1	1912	66	0.0154	0.3729	NaN	NaN	NaN	ImageMagick	
7	46	0	0.0417	0.0000	NaN	NaN	NaN	libvips	
13	1794	28	0.0044	0.2800	NaN	NaN	NaN	php	
14	32	1	0.0000	0.3333	NaN	NaN	NaN	polkit	
16	1887	16	0.0042	0.2286	NaN	NaN	NaN	savannah	
5	85	1	0.0000	0.2500	NaN	NaN	NaN	gstreamer	
15	2918	18	0.0027	0.1552	NaN	NaN	NaN	qemu	
3	47	1	0.0000	0.1667	NaN	NaN	NaN	abrt	
6	588	2	0.0233	0.0211	NaN	NaN	NaN	jasper	

	vulnerable_examples	benign_examples	unique_cwes
8	0	30	1
17	1	11	1
9	0	7	1
0	401	4886	22
18	2	38	1
10	1	15	1
1	177	1942	21
7	3	48	2
13	100	1802	16
14	3	32	1
16	70	1895	18
5	4	85	4
15	116	2926	21
3	6	47	2
6	95	602	8

Highest FP count projects (unseen_val)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	\
0	5287	0.0758	0.7314	0.2117	0.9327	0.3450	374	1393	
1	2119	0.0835	0.9547	0.7872	0.6271	0.6981	111	30	
6	697	0.1363	0.9770	0.8692	0.9789	0.9208	93	14	
15	3042	0.0381	0.9915	0.9245	0.8448	0.8829	98	8	
16	1965	0.0356	0.9878	0.8710	0.7714	0.8182	54	8	
13	1902	0.0526	0.9811	0.9000	0.7200	0.8000	72	8	
7	51	0.0588	0.9608	0.6000	1.0000	0.7500	3	2	
10	16	0.0625	0.9375	0.5000	1.0000	0.6667	1	1	
5	89	0.0449	0.9888	1.0000	0.7500	0.8571	3	0	
11	81	0.0123	1.0000	1.0000	1.0000	1.0000	1	0	
3	53	0.1132	0.9811	1.0000	0.8333	0.9091	5	0	
18	40	0.0500	0.9750	1.0000	0.5000	0.6667	1	0	
2	36	0.0556	1.0000	1.0000	1.0000	1.0000	2	0	
14	35	0.0857	0.9714	1.0000	0.6667	0.8000	2	0	
8	30	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	

	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project	\
0	3493	27	0.2851	0.0673	NaN	NaN	NaN	Android	
1	1912	66	0.0154	0.3729	NaN	NaN	NaN	ImageMagick	
6	588	2	0.0233	0.0211	NaN	NaN	NaN	jasper	
15	2918	18	0.0027	0.1552	NaN	NaN	NaN	qemu	
16	1887	16	0.0042	0.2286	NaN	NaN	NaN	savannah	
13	1794	28	0.0044	0.2800	NaN	NaN	NaN	php	
7	46	0	0.0417	0.0000	NaN	NaN	NaN	libvips	
10	14	0	0.0667	0.0000	NaN	NaN	NaN	netdata	
5	85	1	0.0000	0.2500	NaN	NaN	NaN	gstreamer	
11	80	0	0.0000	0.0000	NaN	NaN	NaN	openvpn	
3	47	1	0.0000	0.1667	NaN	NaN	NaN	abrt	
18	38	1	0.0000	0.5000	NaN	NaN	NaN	zfs	
2	34	0	0.0000	0.0000	NaN	NaN	NaN	NetworkManager	
14	32	1	0.0000	0.3333	NaN	NaN	NaN	polkit	
8	30	0	0.0000	NaN	NaN	NaN	NaN	lynx-snapshots	

	vulnerable_examples	benign_examples	unique_cwes
0	401	4886	22
1	177	1942	21
6	95	602	8
15	116	2926	21
16	70	1895	18
13	100	1802	16
7	3	48	2
10	1	15	1
5	4	85	4
11	1	80	2
3	6	47	2
18	2	38	1
2	2	34	2

14	3	32	1
8	0	30	1

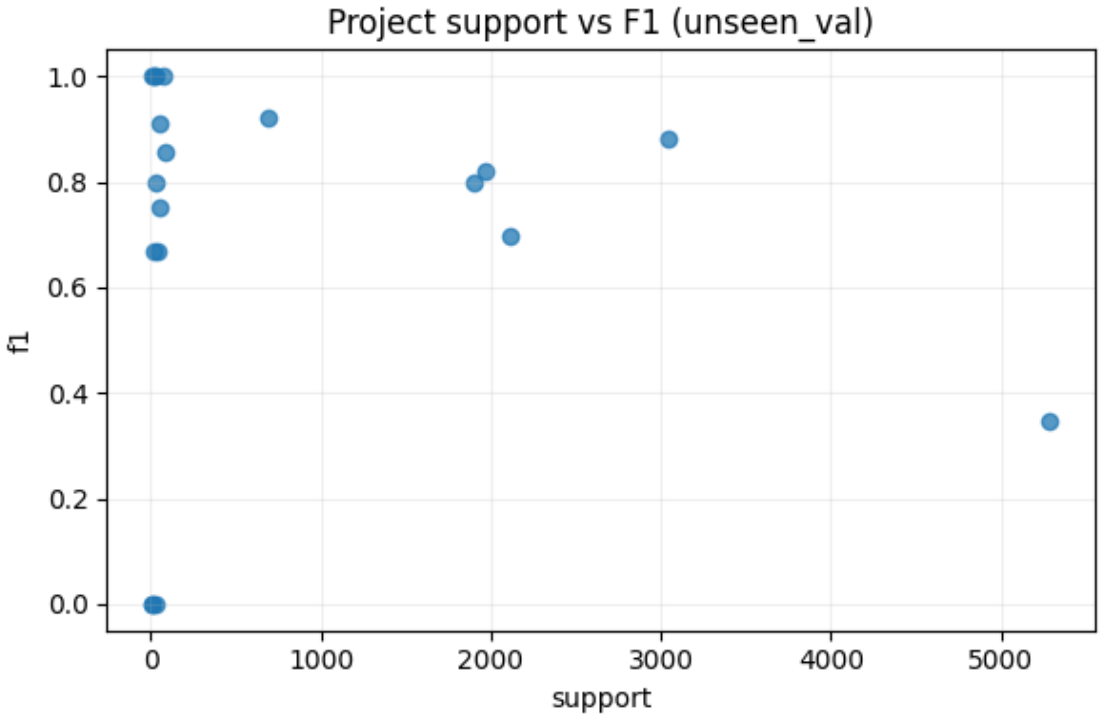
Highest FN count projects (unseen_val)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	\
1	2119	0.0835	0.9547	0.7872	0.6271	0.6981	111	30	
13	1902	0.0526	0.9811	0.9000	0.7200	0.8000	72	8	
0	5287	0.0758	0.7314	0.2117	0.9327	0.3450	374	1393	
15	3042	0.0381	0.9915	0.9245	0.8448	0.8829	98	8	
16	1965	0.0356	0.9878	0.8710	0.7714	0.8182	54	8	
6	697	0.1363	0.9770	0.8692	0.9789	0.9208	93	14	
5	89	0.0449	0.9888	1.0000	0.7500	0.8571	3	0	
3	53	0.1132	0.9811	1.0000	0.8333	0.9091	5	0	
18	40	0.0500	0.9750	1.0000	0.5000	0.6667	1	0	
14	35	0.0857	0.9714	1.0000	0.6667	0.8000	2	0	
17	12	0.0833	0.9167	0.0000	0.0000	0.0000	0	0	
11	81	0.0123	1.0000	1.0000	1.0000	1.0000	1	0	
7	51	0.0588	0.9608	0.6000	1.0000	0.7500	3	2	
2	36	0.0556	1.0000	1.0000	1.0000	1.0000	2	0	
8	30	0.0000	1.0000	0.0000	0.0000	0.0000	0	0	

	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project	\
1	1912	66	0.0154	0.3729	NaN	NaN	NaN	ImageMagick	
13	1794	28	0.0044	0.2800	NaN	NaN	NaN	php	
0	3493	27	0.2851	0.0673	NaN	NaN	NaN	Android	
15	2918	18	0.0027	0.1552	NaN	NaN	NaN	qemu	
16	1887	16	0.0042	0.2286	NaN	NaN	NaN	savannah	
6	588	2	0.0233	0.0211	NaN	NaN	NaN	jasper	
5	85	1	0.0000	0.2500	NaN	NaN	NaN	gstreamer	
3	47	1	0.0000	0.1667	NaN	NaN	NaN	abrt	
18	38	1	0.0000	0.5000	NaN	NaN	NaN	zfs	
14	32	1	0.0000	0.3333	NaN	NaN	NaN	polkit	
17	11	1	0.0000	1.0000	NaN	NaN	NaN	yubico-pam	
11	80	0	0.0000	0.0000	NaN	NaN	NaN	openvpn	
7	46	0	0.0417	0.0000	NaN	NaN	NaN	libvips	
2	34	0	0.0000	0.0000	NaN	NaN	NaN	NetworkManager	
8	30	0	0.0000	NaN	NaN	NaN	NaN	lynx-snapshots	

	vulnerable_examples	benign_examples	unique_cwes
1	177	1942	21
13	100	1802	16
0	401	4886	22
15	116	2926	21
16	70	1895	18
6	95	602	8
5	4	85	4
3	6	47	2

18	2	38	1
14	3	32	1
17	1	11	1
11	1	80	2
7	3	48	2
2	2	34	2
8	0	30	1



1.8.4 Per-Project: unseen_test

Largest projects (unseen_test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	\
4	1737	0.0345	0.9937	0.9153	0.9000	0.9076	54	5	
0	1695	0.0490	0.9876	0.9429	0.7952	0.8627	66	4	
22	1456	0.1422	0.9794	0.9733	0.8792	0.9239	182	5	
21	1419	0.0648	0.9803	0.9103	0.7717	0.8353	71	7	
23	1101	0.0509	0.9855	0.8846	0.8214	0.8519	46	6	
12	668	0.0299	0.9955	0.9474	0.9000	0.9231	18	1	
7	509	0.0118	0.9980	1.0000	0.8333	0.9091	5	0	
11	493	0.2008	0.9797	1.0000	0.8990	0.9468	89	0	
25	440	0.0818	0.9864	1.0000	0.8333	0.9091	30	0	
27	401	0.0100	0.9950	0.6667	1.0000	0.8000	4	2	
10	388	0.0387	1.0000	1.0000	1.0000	1.0000	15	0	
6	339	0.0118	1.0000	1.0000	1.0000	1.0000	4	0	

1	337	0.0475	0.9703	0.7143	0.6250	0.6667	10	4
26	316	0.0759	0.9684	0.9375	0.6250	0.7500	15	1
18	315	0.0190	0.9968	0.8571	1.0000	0.9231	6	1

	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project
4	1672	6	0.0030	0.1000	NaN	NaN	NaN	ghostscript
0	1608	17	0.0025	0.2048	NaN	NaN	NaN	FFmpeg
22	1244	25	0.0040	0.1208	NaN	NaN	NaN	php-src
21	1320	21	0.0053	0.2283	NaN	NaN	NaN	openssl
23	1039	10	0.0057	0.1786	NaN	NaN	NaN	radare2
12	647	2	0.0015	0.1000	NaN	NaN	NaN	libarchive
7	503	1	0.0000	0.1667	NaN	NaN	NaN	httpd
11	394	10	0.0000	0.1010	NaN	NaN	NaN	krb5
25	404	6	0.0000	0.1667	NaN	NaN	NaN	samba
27	395	0	0.0050	0.0000	NaN	NaN	NaN	tor
10	373	0	0.0000	0.0000	NaN	NaN	NaN	keepalived
6	335	0	0.0000	0.0000	NaN	NaN	NaN	haproxy
1	317	6	0.0125	0.3750	NaN	NaN	NaN	ImageMagick6
26	291	9	0.0034	0.3750	NaN	NaN	NaN	src
18	308	0	0.0032	0.0000	NaN	NaN	NaN	mruby

	vulnerable_examples	benign_examples	unique_cwes
4	60	1677	11
0	83	1612	15
22	207	1249	17
21	92	1327	19
23	56	1045	6
12	20	648	11
7	6	503	4
11	99	394	13
25	36	404	10
27	4	397	4
10	15	373	3
6	4	335	5
1	16	321	6
26	24	292	7
18	6	309	5

Worst projects by F1 (unseen_test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	TN
29	102	0.0098	0.9706	0.2500	1.0000	0.4000	1	3	98
9	212	0.0330	0.9575	0.4286	0.8571	0.5714	6	8	197
19	257	0.0195	0.9844	0.6000	0.6000	0.6000	3	2	250
1	337	0.0475	0.9703	0.7143	0.6250	0.6667	10	4	317
28	272	0.0074	0.9963	1.0000	0.5000	0.6667	1	0	270
26	316	0.0759	0.9684	0.9375	0.6250	0.7500	15	1	291
3	233	0.0386	0.9785	0.6667	0.8889	0.7619	8	4	220

20	198	0.1364	0.9495	1.0000	0.6296	0.7727	17	0	171
27	401	0.0100	0.9950	0.6667	1.0000	0.8000	4	2	395
14	108	0.0278	0.9907	1.0000	0.6667	0.8000	2	0	105
21	1419	0.0648	0.9803	0.9103	0.7717	0.8353	71	7	1320
23	1101	0.0509	0.9855	0.8846	0.8214	0.8519	46	6	1039
0	1695	0.0490	0.9876	0.9429	0.7952	0.8627	66	4	1608
30	209	0.0766	0.9809	0.9286	0.8125	0.8667	13	1	192
24	138	0.0507	0.9855	0.7778	1.0000	0.8750	7	2	129

	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project	\
29	0	0.0297	0.0000	NaN	NaN	NaN	wolfssl	
9	1	0.0390	0.1429	NaN	NaN	NaN	kde	
19	2	0.0079	0.4000	NaN	NaN	NaN	oniguruma	
1	6	0.0125	0.3750	NaN	NaN	NaN	ImageMagick6	
28	1	0.0000	0.5000	NaN	NaN	NaN	udisks	
26	9	0.0034	0.3750	NaN	NaN	NaN	src	
3	1	0.0179	0.1111	NaN	NaN	NaN	exempi	
20	10	0.0000	0.3704	NaN	NaN	NaN	openjpeg	
27	0	0.0050	0.0000	NaN	NaN	NaN	tor	
14	1	0.0000	0.3333	NaN	NaN	NaN	libreport	
21	21	0.0053	0.2283	NaN	NaN	NaN	openssl	
23	10	0.0057	0.1786	NaN	NaN	NaN	radare2	
0	17	0.0025	0.2048	NaN	NaN	NaN	FFmpeg	
30	3	0.0052	0.1875	NaN	NaN	NaN	yara	
24	0	0.0153	0.0000	NaN	NaN	NaN	raptor	

	vulnerable_examples	benign_examples	unique_cwes
29	1	101	1
9	7	205	4
19	5	252	5
1	16	321	6
28	2	270	1
26	24	292	7
3	9	224	6
20	27	171	8
27	4	397	4
14	3	105	1
21	92	1327	19
23	56	1045	6
0	83	1612	15
30	16	193	5
24	7	131	1

Highest FP count projects (unseen_test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	\
9	212	0.0330	0.9575	0.4286	0.8571	0.5714	6	8	
21	1419	0.0648	0.9803	0.9103	0.7717	0.8353	71	7	

23	1101	0.0509	0.9855	0.8846	0.8214	0.8519	46	6
4	1737	0.0345	0.9937	0.9153	0.9000	0.9076	54	5
22	1456	0.1422	0.9794	0.9733	0.8792	0.9239	182	5
0	1695	0.0490	0.9876	0.9429	0.7952	0.8627	66	4
1	337	0.0475	0.9703	0.7143	0.6250	0.6667	10	4
3	233	0.0386	0.9785	0.6667	0.8889	0.7619	8	4
29	102	0.0098	0.9706	0.2500	1.0000	0.4000	1	3
27	401	0.0100	0.9950	0.6667	1.0000	0.8000	4	2
19	257	0.0195	0.9844	0.6000	0.6000	0.6000	3	2
24	138	0.0507	0.9855	0.7778	1.0000	0.8750	7	2
12	668	0.0299	0.9955	0.9474	0.9000	0.9231	18	1
26	316	0.0759	0.9684	0.9375	0.6250	0.7500	15	1
18	315	0.0190	0.9968	0.8571	1.0000	0.9231	6	1

	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project	\
9	197	1	0.0390	0.1429	NaN	NaN	NaN	kde	
21	1320	21	0.0053	0.2283	NaN	NaN	NaN	openssl	
23	1039	10	0.0057	0.1786	NaN	NaN	NaN	radare2	
4	1672	6	0.0030	0.1000	NaN	NaN	NaN	ghostscript	
22	1244	25	0.0040	0.1208	NaN	NaN	NaN	php-src	
0	1608	17	0.0025	0.2048	NaN	NaN	NaN	FFmpeg	
1	317	6	0.0125	0.3750	NaN	NaN	NaN	ImageMagick6	
3	220	1	0.0179	0.1111	NaN	NaN	NaN	exempi	
29	98	0	0.0297	0.0000	NaN	NaN	NaN	wolfssl	
27	395	0	0.0050	0.0000	NaN	NaN	NaN	tor	
19	250	2	0.0079	0.4000	NaN	NaN	NaN	oniguruma	
24	129	0	0.0153	0.0000	NaN	NaN	NaN	raptor	
12	647	2	0.0015	0.1000	NaN	NaN	NaN	libarchive	
26	291	9	0.0034	0.3750	NaN	NaN	NaN	src	
18	308	0	0.0032	0.0000	NaN	NaN	NaN	mruby	

	vulnerable_examples	benign_examples	unique_cwes
9	7	205	4
21	92	1327	19
23	56	1045	6
4	60	1677	11
22	207	1249	17
0	83	1612	15
1	16	321	6
3	9	224	6
29	1	101	1
27	4	397	4
19	5	252	5
24	7	131	1
12	20	648	11
26	24	292	7
18	6	309	5

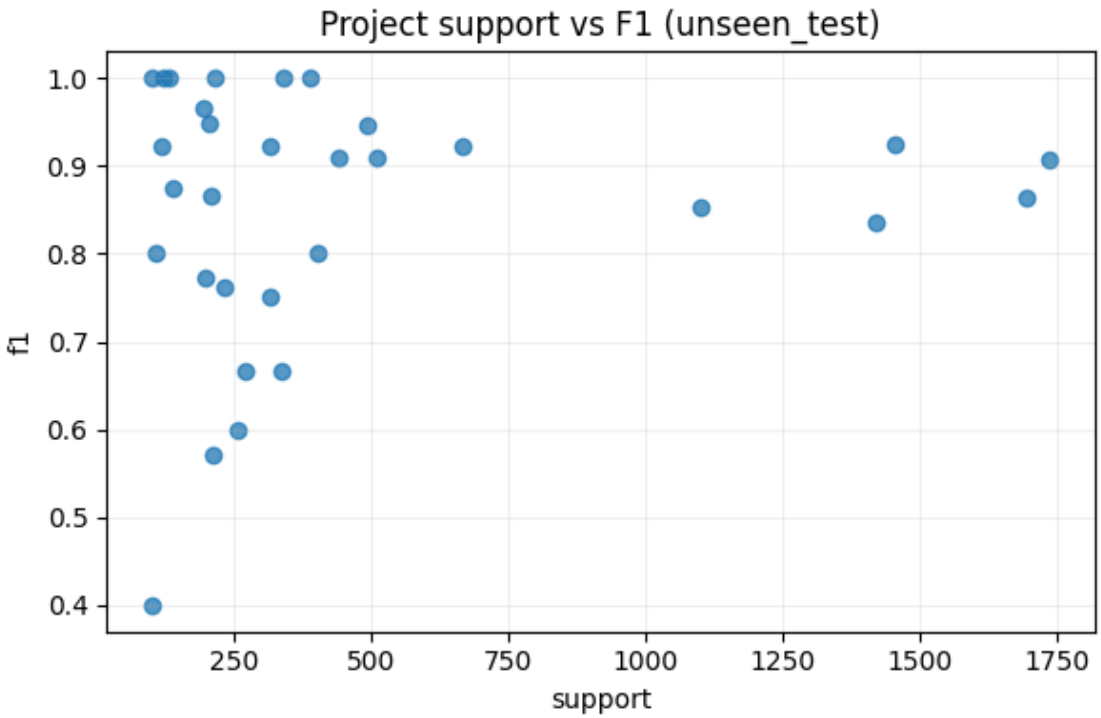
Highest FN count projects (unseen_test)

	support	positive_rate	accuracy	precision	recall	f1	TP	FP	\
22	1456	0.1422	0.9794	0.9733	0.8792	0.9239	182	5	
21	1419	0.0648	0.9803	0.9103	0.7717	0.8353	71	7	
0	1695	0.0490	0.9876	0.9429	0.7952	0.8627	66	4	
23	1101	0.0509	0.9855	0.8846	0.8214	0.8519	46	6	
11	493	0.2008	0.9797	1.0000	0.8990	0.9468	89	0	
20	198	0.1364	0.9495	1.0000	0.6296	0.7727	17	0	
26	316	0.0759	0.9684	0.9375	0.6250	0.7500	15	1	
4	1737	0.0345	0.9937	0.9153	0.9000	0.9076	54	5	
25	440	0.0818	0.9864	1.0000	0.8333	0.9091	30	0	
1	337	0.0475	0.9703	0.7143	0.6250	0.6667	10	4	
30	209	0.0766	0.9809	0.9286	0.8125	0.8667	13	1	
12	668	0.0299	0.9955	0.9474	0.9000	0.9231	18	1	
19	257	0.0195	0.9844	0.6000	0.6000	0.6000	3	2	
7	509	0.0118	0.9980	1.0000	0.8333	0.9091	5	0	
28	272	0.0074	0.9963	1.0000	0.5000	0.6667	1	0	

	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	project	\
22	1244	25	0.0040	0.1208	NaN	NaN	NaN	php-src	
21	1320	21	0.0053	0.2283	NaN	NaN	NaN	openssl	
0	1608	17	0.0025	0.2048	NaN	NaN	NaN	FFmpeg	
23	1039	10	0.0057	0.1786	NaN	NaN	NaN	radare2	
11	394	10	0.0000	0.1010	NaN	NaN	NaN	krb5	
20	171	10	0.0000	0.3704	NaN	NaN	NaN	openjpeg	
26	291	9	0.0034	0.3750	NaN	NaN	NaN	src	
4	1672	6	0.0030	0.1000	NaN	NaN	NaN	ghostscript	
25	404	6	0.0000	0.1667	NaN	NaN	NaN	samba	
1	317	6	0.0125	0.3750	NaN	NaN	NaN	ImageMagick6	
30	192	3	0.0052	0.1875	NaN	NaN	NaN	yara	
12	647	2	0.0015	0.1000	NaN	NaN	NaN	libarchive	
19	250	2	0.0079	0.4000	NaN	NaN	NaN	oniguruma	
7	503	1	0.0000	0.1667	NaN	NaN	NaN	httpd	
28	270	1	0.0000	0.5000	NaN	NaN	NaN	udisks	

	vulnerable_examples	benign_examples	unique_cwes
22	207	1249	17
21	92	1327	19
0	83	1612	15
23	56	1045	6
11	99	394	13
20	27	171	8
26	24	292	7
4	60	1677	11
25	36	404	10
1	16	321	6
30	16	193	5

12	20	648	11
19	5	252	5
7	6	503	4
28	2	270	1



1.8.5 Project x seen/unseen counts (merged eval splits)

is_unseen_cwe	0	1
project		
FFmpeg	0	1695
ImageMagick6	0	337
OpenJK	660	0
curl	0	215
exempi	0	233
ghostscript	0	1737
gnupg	0	204
gpac	1204	0
haproxy	0	339
httpd	0	509
irssi	0	119
kde	0	212
keepalived	0	388
krb5	0	493
libXpm	4	0

```
libXrandr        24     0
libarchive        0   668
libcomps          0   100
libreport         0   108
libxkbcommon      0   122
mbedtls           0   194
miniupnp          0   133
mruby             0   315
oniguruma         0   257
openjpeg          0   198
```

1.8.6 Project x CWE count (merged eval splits)

```
         project  unique_cwes
0        openssl           19
1        php-src           17
2         FFmpeg           15
3           krb5           13
4     libarchive           11
5    ghostscript           11
6          samba           10
7        xserver            9
8       openjpeg            8
9            src            7
10      miniupnp            6
11       poppler            6
12  ImageMagick6            6
13       radare2            6
14        exempi            6
15          curl            6
16     oniguruma            5
17         mruby            5
18       mbedtls            5
19          yara            5
20       haproxy            5
21         gnupg            5
22         httpd            4
23           tor            4
24          gpac            4
```

1.9 Error Analysis

False negatives matter because missed vulnerabilities are operationally costly. False positives matter because they create review burden. Both are broken down by CWE, project, length bucket, and score bucket when possible.

[11]:
```
length_edges = pea.DEFAULT_LENGTH_BUCKETS["token_length"]
fn_df = pea.error_frame(eval_df, "fn")
```

```
fp_df = pea.error_frame(eval_df, "fp")
fn_df = pea.bucket_scores(pea.add_length_buckets(fn_df, "token_length",␣
 ↪length_edges))
fp_df = pea.bucket_scores(pea.add_length_buckets(fp_df, "token_length",␣
 ↪length_edges))

display(Markdown(f"### False negatives: {len(fn_df)} rows"))
display(Markdown(f"Rate within evaluation set: {len(fn_df) / len(eval_df):.
 ↪3f}"))
display(Markdown(f"### False positives: {len(fp_df)} rows"))
display(Markdown(f"Rate within evaluation set: {len(fp_df) / len(eval_df):.
 ↪3f}"))

if "cwe_id" in eval_df.columns:
    display(Markdown("### CWE with highest FN count"))
    display(fn_df.groupby("cwe_id").size().sort_values(ascending=False).
 ↪head(15).rename("fn_count").reset_index())
    display(Markdown("### CWE with highest FP count"))
    display(fp_df.groupby("cwe_id").size().sort_values(ascending=False).
 ↪head(15).rename("fp_count").reset_index())
if "project" in eval_df.columns:
    display(Markdown("### Project with highest FN count"))
    display(fn_df.groupby("project").size().sort_values(ascending=False).
 ↪head(15).rename("fn_count").reset_index())
    display(Markdown("### Project with highest FP count"))
    display(fp_df.groupby("project").size().sort_values(ascending=False).
 ↪head(15).rename("fp_count").reset_index())
if "token_length_bucket" in fn_df.columns:
    display(Markdown("### FN by token-length bucket"))
    display(fn_df.groupby("token_length_bucket", observed=False).size().
 ↪rename("fn_count").reset_index())
    display(Markdown("### FP by token-length bucket"))
    display(fp_df.groupby("token_length_bucket", observed=False).size().
 ↪rename("fp_count").reset_index())
if "score_bucket" in fn_df.columns:
    display(Markdown("### FN by score bucket"))
    display(fn_df.groupby("score_bucket", observed=False).size().
 ↪rename("fn_count").reset_index())
    display(Markdown("### FP by score bucket"))
    display(fp_df.groupby("score_bucket", observed=False).size().
 ↪rename("fp_count").reset_index())

display(Markdown("### Sample false negatives"))
display(pea.sample_error_rows(fn_df, n=20))
display(Markdown("### Sample false positives"))
display(pea.sample_error_rows(fp_df, n=20))
```

1.9.1 False negatives: 151 rows

Rate within evaluation set: 0.008

1.9.2 False positives: 66 rows

Rate within evaluation set: 0.003

1.9.3 CWE with highest FN count

	cwe_id	fn_count
0	CWE-119	26
1	CWE-125	24
2	CWE-20	14
3	CWE-190	12
4	CWE-787	11
5	CWE-476	9
6	CWE-369	8
7	CWE-200	7
8	CWE-189	5
9	CWE-354	4
10	CWE-834	4
11	CWE-310	4
12	CWE-400	3
13	CWE-399	3
14	CWE-416	3

1.9.4 CWE with highest FP count

	cwe_id	fp_count
0	CWE-20	9
1	CWE-200	7
2	CWE-416	7
3	CWE-290	6
4	CWE-476	6
5	CWE-119	4
6	CWE-125	4
7	CWE-190	4
8	CWE-399	4
9	CWE-617	4
10	CWE-787	4
11	CWE-269	3
12	CWE-189	2
13	CWE-311	1
14	CWE-770	1

1.9.5 Project with highest FN count

	project	fn_count
0	php-src	25

1	openssl	21
2	FFmpeg	17
3	radare2	10
4	krb5	10
5	openjpeg	10
6	src	9
7	u-boot	7
8	poppler	7
9	ImageMagick6	6
10	ghostscript	6
11	samba	6
12	yara	3
13	oniguruma	2
14	libarchive	2

1.9.6 Project with highest FP count

	project	fp_count
0	kde	8
1	openssl	7
2	radare2	6
3	php-src	5
4	ghostscript	5
5	ImageMagick6	4
6	FFmpeg	4
7	exempi	4
8	OpenJK	3
9	u-boot	3
10	wolfssl	3
11	oniguruma	2
12	poppler	2
13	raptor	2
14	tor	2

1.9.7 FN by token-length bucket

	token_length_bucket	fn_count
0	0-64	11
1	64-128	9
2	128-256	7
3	256-512	124
4	>512	0

1.9.8 FP by token-length bucket

	token_length_bucket	fp_count
0	0-64	16
1	64-128	3
2	128-256	14

```
3             256-512        33
4                >512         0
```

1.9.9 FN by score bucket

```
  score_bucket  fn_count
0      0.0-0.1       142
1      0.1-0.2         4
2      0.2-0.3         1
3      0.3-0.4         3
4      0.4-0.5         1
5      0.5-0.6         0
6      0.6-0.7         0
7      0.7-0.8         0
8      0.8-0.9         0
9      0.9-1.0         0
```

1.9.10 FP by score bucket

```
  score_bucket  fp_count
0      0.0-0.1         0
1      0.1-0.2         0
2      0.2-0.3         0
3      0.3-0.4         0
4      0.4-0.5         0
5      0.5-0.6         4
6      0.6-0.7         4
7      0.7-0.8         4
8      0.8-0.9         9
9      0.9-1.0        45
```

1.9.11 Sample false negatives

```
       row_id      project   cwe_id  y_true  y_pred         score  \
4617    73054      poppler  CWE-189       1       0  3.443076e-07
4630    73067      poppler  CWE-189       1       0  4.189728e-08
4631    73068      poppler  CWE-189       1       0  1.925102e-09
4634    73071      poppler  CWE-189       1       0  6.388034e-09
4645    73263      poppler   CWE-20       1       0  2.906781e-07
4660    73576      poppler  CWE-119       1       0  4.261700e-09
4664    73645      poppler   CWE-20       1       0  1.204954e-02
4688    75576         gpac  CWE-125       1       0  8.554456e-07
4689    75577         gpac  CWE-125       1       0  7.344977e-11
4694    75863       u-boot  CWE-787       1       0  8.353361e-02
4697    75866       u-boot  CWE-787       1       0  1.770362e-11
4698    75867       u-boot  CWE-787       1       0  4.251050e-12
4699    75868       u-boot  CWE-787       1       0  1.315985e-12
4700    75869       u-boot  CWE-787       1       0  1.091566e-12
4701    75870       u-boot  CWE-787       1       0  3.255889e-12
```

4702	75871	u-boot	CWE-787	1	0	1.316449e-01
18287	73028	samba	CWE-264	1	0	7.923955e-06
18288	73029	openssl	CWE-310	1	0	3.339555e-06
18289	73035	ghostscript	CWE-20	1	0	1.354965e-08
18304	73050	samba	CWE-399	1	0	1.182197e-06

	token_length	code_preview
4617	512	void SplashOutputDev::drawImage(GfxState *stat…
4630	512	void SplashOutputDev::drawMaskedImage(GfxState…
4631	512	void SplashOutputDev::drawSoftMaskedImage(GfxS…
4634	512	GBool SplashFTFont::makeGlyph(int c, int xFrac…
4645	512	void FoFiType1::parse() { char *line, *line1, …
4660	512	void JBIG2Stream::readSegments() { Guint segNu…
4664	168	void DCTStream::reset() { int row_stride; str-…
4688	512	static void oinf_entry_dump(GF_OperatingPoints…
4689	512	GF_Err sgpd_dump(GF_Box *a, FILE * trace) { u3…
4694	188	int fpga_reset(void) { if (!check_boco2()) { /…
4697	36	int get_scl(void) { return qrio_get_gpio(DEBLO…
4698	36	int get_sda(void) { return qrio_get_gpio(DEBLO…
4699	136	static void i2c_deblock_gpio_cfg(void) { /* se…
4700	41	void set_scl(int state) { qrio_set_opendrain_g…
4701	41	void set_sda(int state) { qrio_set_opendrain_g…
4702	49	static void toggle_fpga_eeprom_bus(bool cpu_ow…
18287	512	static int samldb_check_user_account_control_a…
18288	113	long ssl_get_algorithm2(SSL *s) { long alg2 = …
18289	294	void fz_init_cached_color_converter(fz_context…
18304	512	static bool decode_openldap_dereference(void *…

1.9.12 Sample false positives

	row_id	project	cwe_id	y_true	y_pred	score	token_length	\
180	815	poppler	CWE-189	0	1	0.999828	512	
656	3639	poppler	CWE-119	0	1	0.999984	499	
1292	12448	xserver	CWE-20	0	1	0.998939	203	
2656	43179	gpac	CWE-125	0	1	0.999994	512	
3663	50167	u-boot	CWE-787	0	1	0.939602	136	
3668	50172	u-boot	CWE-787	0	1	0.999651	147	
3771	50275	u-boot	CWE-787	0	1	0.835213	413	
4005	54362	OpenJK	CWE-269	0	1	0.962676	512	
4096	54453	OpenJK	CWE-269	0	1	0.704431	215	
4279	54636	OpenJK	CWE-269	0	1	0.522462	25	
5674	2693	ghostscript	CWE-416	0	1	0.823448	387	
6439	5671	kde	CWE-20	0	1	0.830954	63	
6630	6211	kde	CWE-290	0	1	0.981753	43	
6646	6227	kde	CWE-290	0	1	0.890070	39	
6653	6234	kde	CWE-290	0	1	0.938213	33	
6655	6236	kde	CWE-290	0	1	0.999998	120	
6663	6244	kde	CWE-290	0	1	0.990674	43	

```
6674    6255          kde  CWE-290       0       1  0.955216           33
6871    8278      openssl  CWE-399       0       1  0.919280          512
7142    9576          kde  CWE-200       0       1  0.981702           64
```

```
                                                code_preview
180   void CairoOutputDev::updateFont(GfxState *stat…
656   Guchar *ImageStream::getLine() { Gulong buf, b…
1292  PanoramiXRenderFreePicture (ClientPtr client) …
2656  GF_Err audio_sample_entry_AddBox(GF_Box *s, GF…
3663  void qrio_uprstreq(u8 mode) { u32 rstcfg; void…
3668  static int fpga_done(void) { int ret = 0; u8 r…
3771  int trace_list_calls(void *buff, size_t buff_s…
4005  void Com_Frame( void ) { int msec, minMsec; in…
4096  qboolean S_AL_SrcInit( void ) { int i; int lim…
4279    void CL_StopVideo_f( void ) { CL_CloseAVI( ); }
5674  pdf14_buf_free(pdf14_buf *buf) { pdf14_parent_…
6439  Smb4KMountJob::Smb4KMountJob( QObject *parent …
6630  bool AuthBackend::actionExists(const QString &…
6646  bool PolicyKitBackend::isCallerAuthorized(cons…
6653  QByteArray Polkit1Backend::callerID() const { …
6655  bool Polkit1Backend::isCallerAuthorized(const …
6663  bool AuthBackend::actionExists(const QString& …
6674  QByteArray Polkit1Backend::callerID() const { …
6871  static int tls_decrypt_ticket(SSL *s, const un…
7142  NotificationsEngine::NotificationsEngine( QObj…
```

1.10 Length Stratification

If performance degrades in long-function buckets, the model may be struggling near truncation or with structurally complex functions.

[12]:
```
length_tables = {}
for column, edges in pea.DEFAULT_LENGTH_BUCKETS.items():
    table = pea.stratified_metrics(eval_df, column, edges)
    length_tables[column] = table
    if table.empty:
        print(f"Skipping {column}: column not available.")
        continue
    display(Markdown(f"### {column} stratification"))
    display(table.round(4))
    pea.save_dataframe(table, OUTPUT_DIR / f"{column}_stratified_metrics.csv")

if "cwe_id" in eval_df.columns:
    rarity_df = eval_df.copy()
    rarity_df["rare_cwe"] = pea.rarity_flags(rarity_df, "cwe_id",␣
 ↪min_support=20)
    display(Markdown("### Rare vs frequent CWE"))
    display(pea.grouped_metrics(rarity_df, "rare_cwe").round(4))
```

```
if "project" in eval_df.columns:
    rarity_df = eval_df.copy()
    rarity_df["rare_project"] = pea.rarity_flags(rarity_df, "project",␣
 ↪min_support=20)
    display(Markdown("### Rare vs frequent project"))
    display(pea.grouped_metrics(rarity_df, "rare_project").round(4))
```

```
/Users/noamc/repos/graph-vul/graph_vul/analysis/post_experiment_analysis.py:372:
FutureWarning: The default of observed=False is deprecated and will be changed
to True in a future version of pandas. Pass observed=False to retain current
behavior or observed=True to adopt the future default and silence this warning.
  for group_value, group_df in df.groupby(group_col, dropna=False):
```

1.10.1 token_length stratification

```
   support  positive_rate  accuracy  precision  recall      f1   TP  FP    TN  \
3     7313         0.0948    0.9785     0.9452  0.8211  0.8788  569  33  6587
2     4871         0.0357    0.9957     0.9227  0.9598  0.9408  167  14  4683
1     3981         0.0224    0.9970     0.9639  0.8989  0.9302   80   3  3889
0     2992         0.0084    0.9910     0.4667  0.5600  0.5091   14  16  2951
4        0            NaN       NaN        NaN     NaN     NaN    0   0     0

    FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score token_length_bucket
3  124   0.0050   0.1789      NaN     NaN          NaN             256-512
2    7   0.0030   0.0402      NaN     NaN          NaN             128-256
1    9   0.0008   0.1011      NaN     NaN          NaN              64-128
0   11   0.0054   0.4400      NaN     NaN          NaN                0-64
4    0      NaN      NaN      NaN     NaN          NaN                >512
```

```
/Users/noamc/repos/graph-vul/graph_vul/analysis/post_experiment_analysis.py:372:
FutureWarning: The default of observed=False is deprecated and will be changed
to True in a future version of pandas. Pass observed=False to retain current
behavior or observed=True to adopt the future default and silence this warning.
  for group_value, group_df in df.groupby(group_col, dropna=False):
```

1.10.2 line_count stratification

```
   support  positive_rate  accuracy  precision  recall      f1   TP  FP  \
0    13336         0.0280    0.9949     0.9003  0.9196  0.9098  343  38
1     3492         0.0845    0.9905     0.9517  0.9356  0.9436  276  14
2     1976         0.1215    0.9590     0.9392  0.7083  0.8076  170  11
3      350         0.2029    0.9057     0.9318  0.5775  0.7130   41   3
4        3         0.6667    0.3333     0.0000  0.0000  0.0000    0   0

      TN  FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score line_count_bucket
0  12925  30   0.0029   0.0804      NaN     NaN          NaN              0-32
1   3183  19   0.0044   0.0644      NaN     NaN          NaN             32-64
2   1725  70   0.0063   0.2917      NaN     NaN          NaN            64-128
```

```
3    276  30   0.0108   0.4225      NaN     NaN          NaN           128-256
4      1   2   0.0000   1.0000      NaN     NaN          NaN              >256
```

```
/Users/noamc/repos/graph-vul/graph_vul/analysis/post_experiment_analysis.py:372:
FutureWarning: The default of observed=False is deprecated and will be changed
to True in a future version of pandas. Pass observed=False to retain current
behavior or observed=True to adopt the future default and silence this warning.
  for group_value, group_df in df.groupby(group_col, dropna=False):
```

1.10.3 char_length stratification

```
   support  positive_rate  accuracy  precision  recall      f1   TP  FP    TN  \
0     5453         0.0139    0.9934     0.7632  0.7632  0.7632   58  18  5359
1     4623         0.0327    0.9957     0.9281  0.9404  0.9342  142  11  4461
2     4040         0.0490    0.9960     0.9505  0.9697  0.9600  192  10  3832
3     2989         0.0880    0.9896     0.9531  0.9278  0.9403  244  12  2714
4     1706         0.1331    0.9549     0.9213  0.7225  0.8099  164  14  1465
5      346         0.1908    0.8931     0.9677  0.4545  0.6186   30   1   279

   FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score char_length_bucket
0  18   0.0033   0.2368      NaN     NaN          NaN              0-256
1   9   0.0025   0.0596      NaN     NaN          NaN            256-512
2   6   0.0026   0.0303      NaN     NaN          NaN           512-1024
3  19   0.0044   0.0722      NaN     NaN          NaN          1024-2048
4  63   0.0095   0.2775      NaN     NaN          NaN          2048-4096
5  36   0.0036   0.5455      NaN     NaN          NaN              >4096
```

1.10.4 Rare vs frequent CWE

```
   support  positive_rate  accuracy  precision  recall      f1   TP  FP  \
0    19047         0.0510    0.9888     0.9258  0.8477  0.8851  824  66
1      110         0.0818    0.9727     1.0000  0.6667  0.8000    6   0

      TN   FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score  rare_cwe
0  18009  148   0.0037   0.1523      NaN     NaN          NaN     False
1    101    3   0.0000   0.3333      NaN     NaN          NaN      True
```

1.10.5 Rare vs frequent project

```
   support  positive_rate  accuracy  precision  recall      f1   TP  FP  \
0    19153         0.0512    0.9887     0.9263  0.8459  0.8843  829  66
1        4         0.2500    1.0000     1.0000  1.0000  1.0000    1   0

      TN   FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score  rare_project
0  18107  151   0.0036   0.1541      NaN     NaN          NaN         False
1      3    0   0.0000   0.0000      NaN     NaN          NaN          True
```

1.11 Threshold Analysis

Threshold tuning matters when the model shows high precision with lower recall. Threshold sweep and tuning use only `val` / `unseen_val` — never `test` or `unseen_test`. The tuned threshold is then applied to test splits for comparison display only.

[13]:
```
effective_score_kind = next((meta.get("score_kind") for meta in load_metadata.
 ↪values() if meta.get("has_score")), None)
effective_score_kind = None if effective_score_kind == "unknown" else␣
 ↪effective_score_kind
threshold_grid = np.array(THRESHOLD_GRID)

def _decision_values(df, score_kind):
    """Return the column to compare against thresholds, respecting score_kind.
 ↪"""
    if score_kind == "logit" and "score_raw" in df.columns:
        return df["score_raw"]
    return df["score"]

# Sweep on val splits ONLY - never on test/unseen_test.
threshold_sweeps = {}
for split_name in ["val", "unseen_val"]:
    if split_name in frames and "score" in frames[split_name].columns:
        sweep = pea.threshold_sweep(frames[split_name], threshold_grid,␣
 ↪score_kind=effective_score_kind)
        if not sweep.empty:
            sweep["split"] = split_name
            threshold_sweeps[split_name] = sweep

if threshold_sweeps:
    threshold_sweep_df = pd.concat(threshold_sweeps.values(), ignore_index=True)
    display(Markdown("### Threshold sweep (validation splits only)"))
    display(threshold_sweep_df.round(4))
    pea.save_dataframe(threshold_sweep_df, OUTPUT_DIR / "threshold_sweep.csv")
    fig = pea.plot_threshold_curves(threshold_sweeps)
    display(fig)
    pea.save_figure(fig, OUTPUT_DIR / "threshold_sensitivity.png")
    plt.close(fig)
else:
    threshold_sweep_df = pd.DataFrame()
    print("Skipping threshold sweep: no validation split with scores available.
 ↪")

# Find best threshold from val, then APPLY to test/unseen_test for comparison␣
 ↪display only.
threshold_note = None
if "val" in frames and "score" in frames["val"].columns:
```

```
    best_threshold = pea.best_threshold_from_validation(frames["val"],␣
↪threshold_grid, score_kind=effective_score_kind)
    display(Markdown("### Best validation threshold"))
    display(pd.DataFrame([best_threshold]).round(4))
    comparison_rows = []
    for split_name in ["val", "test", "unseen_val", "unseen_test"]:
        if split_name not in frames:
            continue
        base_metrics = pea.compute_binary_metrics(frames[split_name])
        tuned = frames[split_name].copy()
        decision_vals = _decision_values(tuned, effective_score_kind)
        tuned["y_pred"] = (decision_vals >= best_threshold["threshold"]).
↪astype(int)
        tuned_metrics = pea.compute_binary_metrics(tuned)
        comparison_rows.append({
            "split": split_name,
            "setting": "default_threshold",
            "threshold": THRESHOLD,
            "precision": base_metrics["precision"],
            "recall": base_metrics["recall"],
            "f1": base_metrics["f1"],
        })
        comparison_rows.append({
            "split": split_name,
            "setting": "val_tuned_threshold",
            "threshold": best_threshold["threshold"],
            "precision": tuned_metrics["precision"],
            "recall": tuned_metrics["recall"],
            "f1": tuned_metrics["f1"],
        })
    threshold_comparison = pd.DataFrame(comparison_rows)
    display(Markdown("### Default vs validation-tuned threshold (tuned on val,␣
↪applied to all splits)"))
    display(threshold_comparison.round(4))
    pea.save_dataframe(threshold_comparison, OUTPUT_DIR / "threshold_comparison.
↪csv")
    threshold_note = (
        f"Validation-based threshold tuning selected␣
↪{best_threshold['threshold']:.2f}. "
        "Compare the tuned threshold against the default setting before␣
↪changing production evaluation defaults."
    )
else:
    display(Markdown("Diagnostic only: validation scores unavailable, so␣
↪threshold curves should not be used to pick the final threshold."))
```

1.11.1 Threshold sweep (validation splits only)

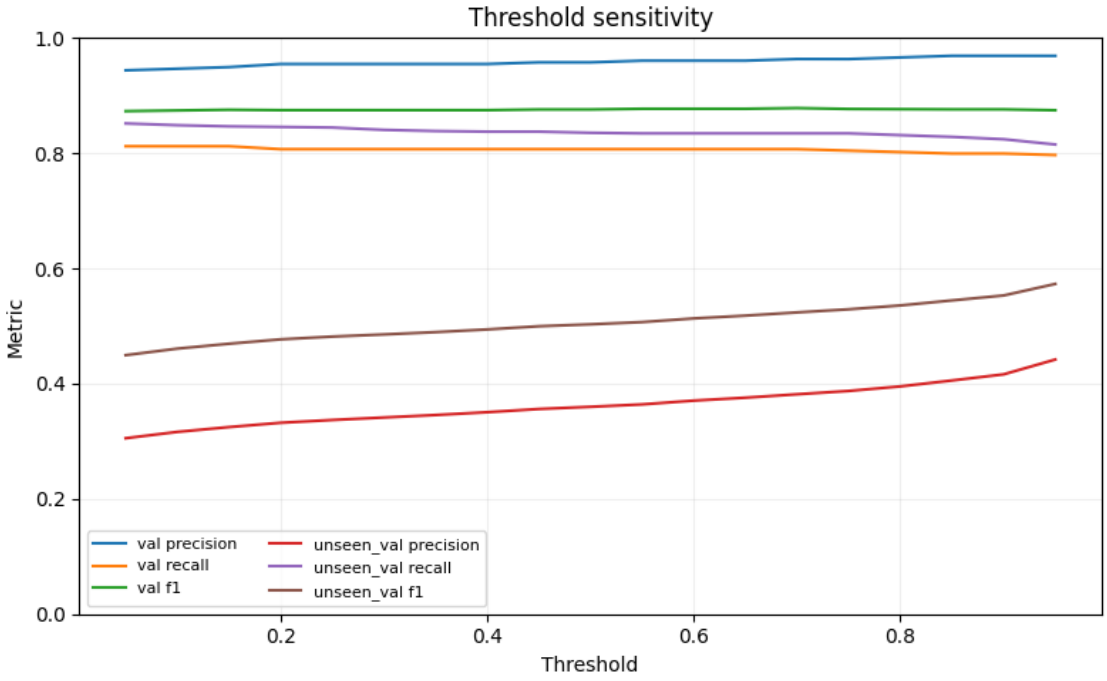
	support	positive_rate	accuracy	precision	recall	f1	TP	FP	\
0	8465	0.0465	0.9890	0.9440	0.8122	0.8731	320	19	
1	8465	0.0465	0.9891	0.9467	0.8122	0.8743	320	18	
2	8465	0.0465	0.9892	0.9496	0.8122	0.8755	320	17	
3	8465	0.0465	0.9892	0.9550	0.8071	0.8748	318	15	
4	8465	0.0465	0.9892	0.9550	0.8071	0.8748	318	15	
5	8465	0.0465	0.9892	0.9550	0.8071	0.8748	318	15	
6	8465	0.0465	0.9892	0.9550	0.8071	0.8748	318	15	
7	8465	0.0465	0.9892	0.9550	0.8071	0.8748	318	15	
8	8465	0.0465	0.9894	0.9578	0.8071	0.8760	318	14	
9	8465	0.0465	0.9894	0.9578	0.8071	0.8760	318	14	
10	8465	0.0465	0.9895	0.9607	0.8071	0.8772	318	13	
11	8465	0.0465	0.9895	0.9607	0.8071	0.8772	318	13	
12	8465	0.0465	0.9895	0.9607	0.8071	0.8772	318	13	
13	8465	0.0465	0.9896	0.9636	0.8071	0.8785	318	12	
14	8465	0.0465	0.9895	0.9635	0.8046	0.8769	317	12	
15	8465	0.0465	0.9895	0.9664	0.8020	0.8766	316	11	
16	8465	0.0465	0.9895	0.9692	0.7995	0.8762	315	10	
17	8465	0.0465	0.9895	0.9692	0.7995	0.8762	315	10	
18	8465	0.0465	0.9894	0.9691	0.7970	0.8747	314	10	
19	15487	0.0636	0.8673	0.3053	0.8518	0.4495	839	1909	
20	15487	0.0636	0.8737	0.3163	0.8487	0.4609	836	1807	
21	15487	0.0636	0.8782	0.3246	0.8467	0.4693	834	1735	
22	15487	0.0636	0.8820	0.3321	0.8457	0.4770	833	1675	
23	15487	0.0636	0.8844	0.3370	0.8447	0.4818	832	1637	
24	15487	0.0636	0.8867	0.3413	0.8406	0.4855	828	1598	
25	15487	0.0636	0.8887	0.3456	0.8386	0.4895	826	1564	
26	15487	0.0636	0.8909	0.3505	0.8376	0.4942	825	1529	
27	15487	0.0636	0.8933	0.3561	0.8376	0.4997	825	1492	
28	15487	0.0636	0.8950	0.3599	0.8355	0.5031	823	1464	
29	15487	0.0636	0.8968	0.3640	0.8345	0.5069	822	1436	
30	15487	0.0636	0.8993	0.3706	0.8345	0.5133	822	1396	
31	15487	0.0636	0.9013	0.3757	0.8345	0.5181	822	1366	
32	15487	0.0636	0.9035	0.3816	0.8345	0.5237	822	1332	
33	15487	0.0636	0.9055	0.3874	0.8345	0.5291	822	1300	
34	15487	0.0636	0.9084	0.3953	0.8315	0.5358	819	1253	
35	15487	0.0636	0.9119	0.4056	0.8284	0.5445	816	1196	
36	15487	0.0636	0.9153	0.4162	0.8244	0.5531	812	1139	
37	15487	0.0636	0.9228	0.4419	0.8152	0.5732	803	1014	

	TN	FN	fp_rate	fn_rate	roc_auc	pr_auc	brier_score	threshold	\
0	8052	74	0.0024	0.1878	NaN	NaN	NaN	0.05	
1	8053	74	0.0022	0.1878	NaN	NaN	NaN	0.10	
2	8054	74	0.0021	0.1878	NaN	NaN	NaN	0.15	
3	8056	76	0.0019	0.1929	NaN	NaN	NaN	0.20	
4	8056	76	0.0019	0.1929	NaN	NaN	NaN	0.25	

5	8056	76	0.0019	0.1929	NaN	NaN	NaN	0.30
6	8056	76	0.0019	0.1929	NaN	NaN	NaN	0.35
7	8056	76	0.0019	0.1929	NaN	NaN	NaN	0.40
8	8057	76	0.0017	0.1929	NaN	NaN	NaN	0.45
9	8057	76	0.0017	0.1929	NaN	NaN	NaN	0.50
10	8058	76	0.0016	0.1929	NaN	NaN	NaN	0.55
11	8058	76	0.0016	0.1929	NaN	NaN	NaN	0.60
12	8058	76	0.0016	0.1929	NaN	NaN	NaN	0.65
13	8059	76	0.0015	0.1929	NaN	NaN	NaN	0.70
14	8059	77	0.0015	0.1954	NaN	NaN	NaN	0.75
15	8060	78	0.0014	0.1980	NaN	NaN	NaN	0.80
16	8061	79	0.0012	0.2005	NaN	NaN	NaN	0.85
17	8061	79	0.0012	0.2005	NaN	NaN	NaN	0.90
18	8061	80	0.0012	0.2030	NaN	NaN	NaN	0.95
19	12593	146	0.1316	0.1482	NaN	NaN	NaN	0.05
20	12695	149	0.1246	0.1513	NaN	NaN	NaN	0.10
21	12767	151	0.1196	0.1533	NaN	NaN	NaN	0.15
22	12827	152	0.1155	0.1543	NaN	NaN	NaN	0.20
23	12865	153	0.1129	0.1553	NaN	NaN	NaN	0.25
24	12904	157	0.1102	0.1594	NaN	NaN	NaN	0.30
25	12938	159	0.1078	0.1614	NaN	NaN	NaN	0.35
26	12973	160	0.1054	0.1624	NaN	NaN	NaN	0.40
27	13010	160	0.1029	0.1624	NaN	NaN	NaN	0.45
28	13038	162	0.1010	0.1645	NaN	NaN	NaN	0.50
29	13066	163	0.0990	0.1655	NaN	NaN	NaN	0.55
30	13106	163	0.0963	0.1655	NaN	NaN	NaN	0.60
31	13136	163	0.0942	0.1655	NaN	NaN	NaN	0.65
32	13170	163	0.0918	0.1655	NaN	NaN	NaN	0.70
33	13202	163	0.0896	0.1655	NaN	NaN	NaN	0.75
34	13249	166	0.0864	0.1685	NaN	NaN	NaN	0.80
35	13306	169	0.0825	0.1716	NaN	NaN	NaN	0.85
36	13363	173	0.0785	0.1756	NaN	NaN	NaN	0.90
37	13488	182	0.0699	0.1848	NaN	NaN	NaN	0.95

	split
0	val
1	val
2	val
3	val
4	val
5	val
6	val
7	val
8	val
9	val
10	val
11	val
12	val

```
13          val
14          val
15          val
16          val
17          val
18          val
19  unseen_val
20  unseen_val
21  unseen_val
22  unseen_val
23  unseen_val
24  unseen_val
25  unseen_val
26  unseen_val
27  unseen_val
28  unseen_val
29  unseen_val
30  unseen_val
31  unseen_val
32  unseen_val
33  unseen_val
34  unseen_val
35  unseen_val
36  unseen_val
37  unseen_val
```



1.11.2 Best validation threshold

```
   support  positive_rate  accuracy  precision  recall      f1     TP    FP  \
0   8465.0         0.0465    0.9896     0.9636  0.8071  0.8785  318.0  12.0

       TN    FN  fp_rate  fn_rate  roc_auc  pr_auc  brier_score  threshold
0  8059.0  76.0   0.0015   0.1929      NaN     NaN          NaN        0.7
```

1.11.3 Default vs validation-tuned threshold (tuned on val, applied to all splits)

```
         split              setting  threshold  precision  recall      f1
0          val    default_threshold        0.5     0.9578  0.8071  0.8760
1          val  val_tuned_threshold        0.7     0.9636  0.8071  0.8785
2         test    default_threshold        0.5     0.9048  0.8559  0.8796
3         test  val_tuned_threshold        0.7     0.9135  0.8559  0.8837
4   unseen_val    default_threshold        0.5     0.3599  0.8355  0.5031
5   unseen_val  val_tuned_threshold        0.7     0.3816  0.8345  0.5237
6  unseen_test    default_threshold        0.5     0.9292  0.8448  0.8850
7  unseen_test  val_tuned_threshold        0.7     0.9370  0.8379  0.8847
```

1.12 Confidence / Score Analysis

This section checks whether false negatives cluster near the threshold and whether unseen positives receive systematically lower scores than seen positives. If that pattern appears, threshold tuning may improve recall without retraining.

[14]:
```
if "score" not in eval_df.columns:
    print("Skipping score distribution analysis: no score/probability/logit␣
 ↪column available.")
else:
    scored_eval = eval_df.dropna(subset=["score"]).copy()
    tp = scored_eval[(scored_eval["y_true"] == 1) & (scored_eval["y_pred"] ==␣
 ↪1)]
    fp = scored_eval[(scored_eval["y_true"] == 0) & (scored_eval["y_pred"] ==␣
 ↪1)]
    tn = scored_eval[(scored_eval["y_true"] == 0) & (scored_eval["y_pred"] ==␣
 ↪0)]
    fn = scored_eval[(scored_eval["y_true"] == 1) & (scored_eval["y_pred"] ==␣
 ↪0)]

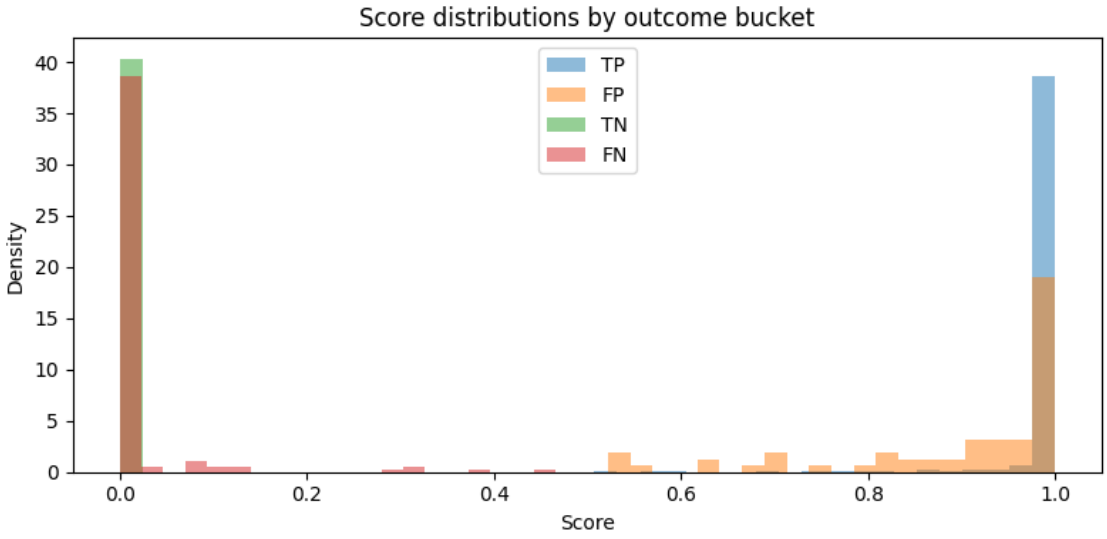
    fig = pea.plot_score_histogram({
        "TP": tp["score"],
        "FP": fp["score"],
        "TN": tn["score"],
        "FN": fn["score"],
    }, "Score distributions by outcome bucket")
    display(fig)
    pea.save_figure(fig, OUTPUT_DIR / "score_distribution_outcomes.png")
```

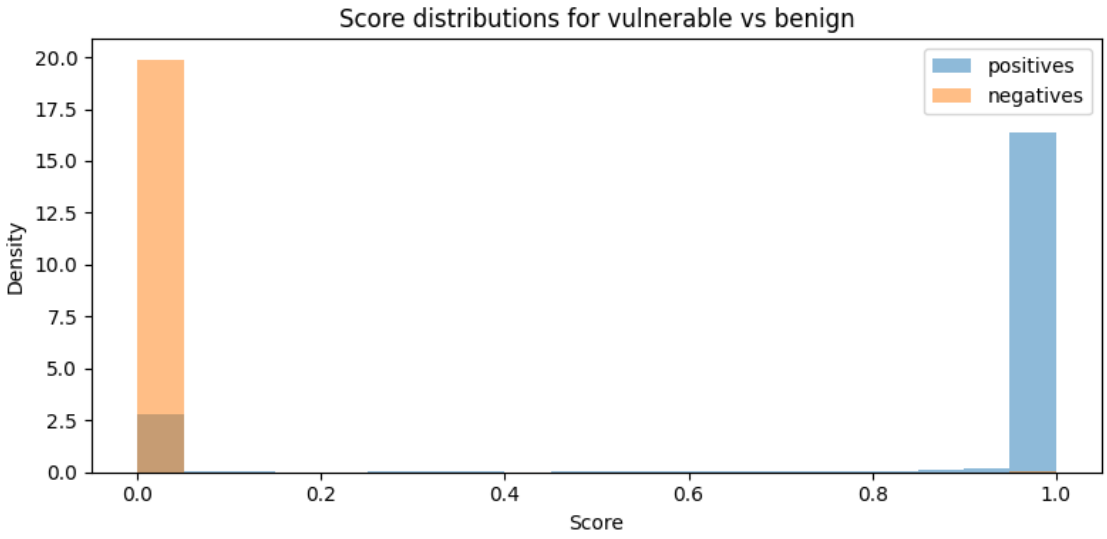
```
    plt.close(fig)

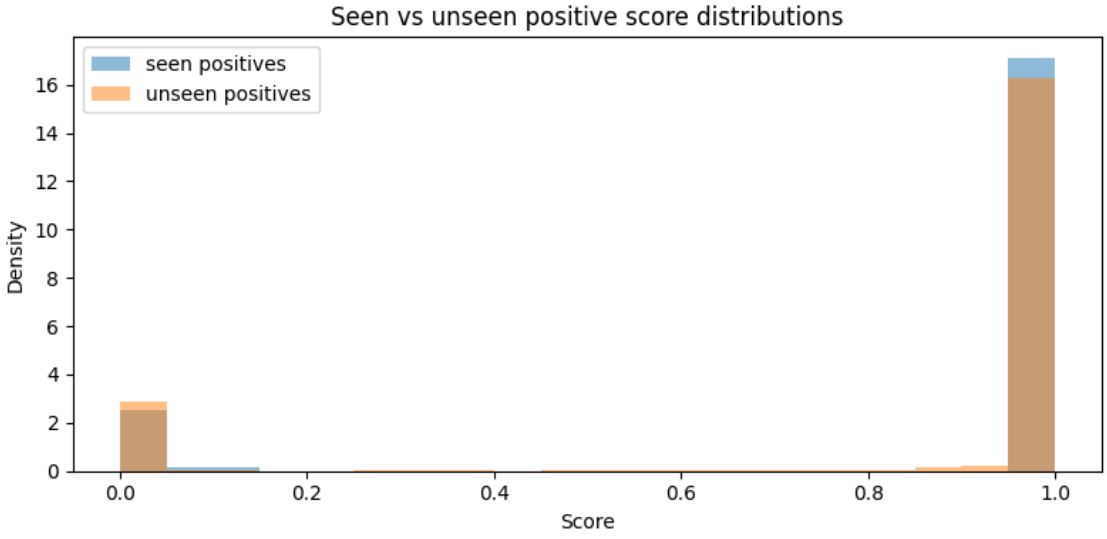
    fig = pea.plot_score_histogram({
        "positives": scored_eval.loc[scored_eval["y_true"] == 1, "score"],
        "negatives": scored_eval.loc[scored_eval["y_true"] == 0, "score"],
    }, "Score distributions for vulnerable vs benign")
    display(fig)
    pea.save_figure(fig, OUTPUT_DIR / "score_distribution_labels.png")
    plt.close(fig)

    if "is_unseen_cwe" in scored_eval.columns:
        fig = pea.plot_score_histogram({
            "seen positives": scored_eval.loc[(scored_eval["y_true"] == 1) &␣
↪(scored_eval["is_unseen_cwe"] == 0), "score"],
            "unseen positives": scored_eval.loc[(scored_eval["y_true"] == 1) &␣
↪(scored_eval["is_unseen_cwe"] == 1), "score"],
        }, "Seen vs unseen positive score distributions")
        display(fig)
        pea.save_figure(fig, OUTPUT_DIR /␣
↪"score_distribution_seen_vs_unseen_positives.png")
        plt.close(fig)

    reliability = pea.reliability_table(scored_eval)
    if not reliability.empty:
        display(Markdown("### Reliability table"))
        display(reliability.round(4))
        pea.save_dataframe(reliability, OUTPUT_DIR / "reliability_table.csv")
        fig = pea.plot_reliability(reliability)
        display(fig)
        pea.save_figure(fig, OUTPUT_DIR / "reliability_plot.png")
        plt.close(fig)
```



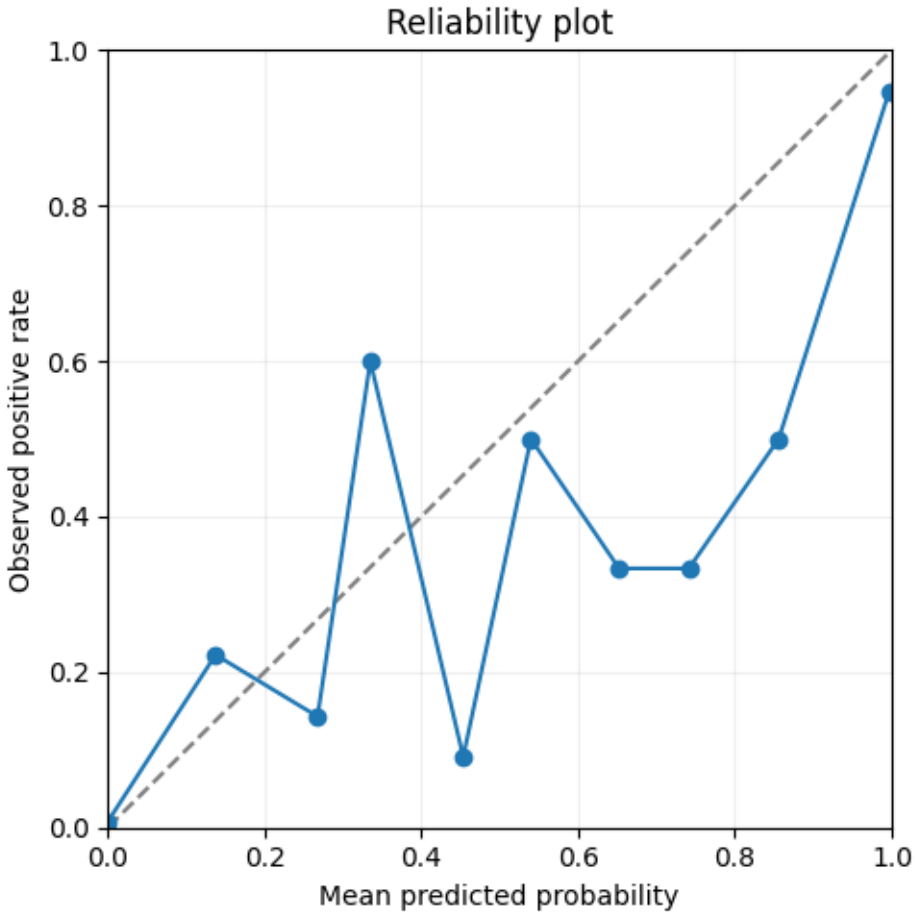




1.12.1 Reliability table

	prob_bin	support	mean_pred	empirical_rate
0	(-0.001, 0.1]	18220	0.0001	0.0078
1	(0.1, 0.2]	18	0.1374	0.2222
2	(0.2, 0.3]	7	0.2677	0.1429
3	(0.3, 0.4]	5	0.3349	0.6
4	(0.4, 0.5]	11	0.4535	0.0909

5	(0.5, 0.6]	8	0.5398	0.5
6	(0.6, 0.7]	6	0.6523	0.3333
7	(0.7, 0.8]	6	0.7413	0.3333
8	(0.8, 0.9]	18	0.8559	0.5
9	(0.9, 1.0]	858	0.9971	0.9476



1.13 Bootstrap Stability

Point estimates are not enough for repeatable comparisons. This section adds bootstrap confidence intervals for `f1`, `precision`, and `recall` on `test` and `unseen_test`.

[15]:
```
bootstrap_df = pea.bootstrap_summary(frames, n_bootstrap=N_BOOTSTRAP)
if bootstrap_df.empty:
    print("Skipping bootstrap summary: test/unseen_test predictions not both␣
 ↪available.")
else:
    display(bootstrap_df.round(4))
    pea.save_dataframe(bootstrap_df, OUTPUT_DIR / "bootstrap_summary.csv")
```

```
delta_rows = []
if {"test", "unseen_test"}.issubset(aggregate_metrics.index):
    for metric_name in ["f1", "precision", "recall"]:
        delta_rows.append({
            "metric": metric_name,
            "test": aggregate_metrics.loc["test", metric_name],
            "unseen_test": aggregate_metrics.loc["unseen_test", metric_name],
            "delta_unseen_minus_test": aggregate_metrics.loc["unseen_test",␣
 ↪metric_name] - aggregate_metrics.loc["test", metric_name],
        })
delta_df = pd.DataFrame(delta_rows)
display(Markdown("### Test vs unseen delta"))
display(delta_df.round(4))
pea.save_dataframe(delta_df, OUTPUT_DIR / "test_vs_unseen_delta.csv")
```

	metric	mean	lower	upper	n_bootstrap	split
0	f1	0.8790	0.8298	0.9218	1000	test
1	precision	0.9046	0.8352	0.9604	1000	test
2	recall	0.8558	0.7870	0.9184	1000	test
3	f1	0.8856	0.8689	0.9006	1000	unseen_test
4	precision	0.9295	0.9118	0.9457	1000	unseen_test
5	recall	0.8457	0.8203	0.8681	1000	unseen_test

1.13.1 Test vs unseen delta

	metric	test	unseen_test	delta_unseen_minus_test
0	f1	0.8796	0.8850	0.0054
1	precision	0.9048	0.9292	0.0244
2	recall	0.8559	0.8448	-0.0110

1.14 Comparison Mode

Optional mode for future runs. Point `COMPARISON_SUMMARY_PATHS` to previously exported `aggregate_metrics.csv` files from this notebook to compare runs consistently.

[16]:
```
COMPARISON_SUMMARY_PATHS = []

comparison_rows = []
for summary_path in COMPARISON_SUMMARY_PATHS:
    summary_path = Path(summary_path)
    if not summary_path.exists():
        print(f"Missing comparison summary: {summary_path}")
        continue
    summary_df = pd.read_csv(summary_path)
    run_label = summary_path.parent.name
    for split_name in ["test", "unseen_test"]:
        split_df = summary_df[summary_df["split"] == split_name]
```

```
        if split_df.empty:
            continue
        row = split_df.iloc[0].to_dict()
        row["run"] = run_label
        comparison_rows.append(row)

comparison_df = pd.DataFrame(comparison_rows)
if comparison_df.empty:
    print("No comparison summaries configured.")
else:
    display(comparison_df[["run", "split", "precision", "recall", "f1",␣
 ↪"support"]].sort_values(["split", "f1"], ascending=[True, False]).round(4))
    pea.save_dataframe(comparison_df, OUTPUT_DIR / "run_comparison.csv")
```

```
No comparison summaries configured.
```

1.15 Final Summary

This is the copy-paste section for reports. It should be concise, stable across runs, and explicit about generalization, conservatism, weak areas, and threshold tuning.

[17]:
```
# Use test or unseen_test per-CWE/per-project for red flags (prefer test)
_summary_per_cwe = next((per_cwe_by_split[s] for s in ["test", "unseen_test"]␣
 ↪if s in per_cwe_by_split), None)
_summary_per_project = next((per_project_by_split[s] for s in ["test",␣
 ↪"unseen_test"] if s in per_project_by_split), None)

red_flags = pea.red_flag_summary(
    aggregate_metrics,
    per_cwe_df=_summary_per_cwe,
    per_project_df=_summary_per_project,
    length_tables=length_tables,
)
report_summary = pea.make_report_summary(
    aggregate_metrics,
    per_cwe_df=_summary_per_cwe,
    per_project_df=_summary_per_project,
    threshold_note=threshold_note,
)
display(Markdown("### Report Summary"))
display(Markdown(report_summary))
summary_path = OUTPUT_DIR / "report_summary.md"
summary_path.write_text(report_summary)

if red_flags:
    display(Markdown("### Red Flags"))
    for flag in red_flags:
        display(Markdown(f"- {flag}"))
```

```
else:
    display(Markdown("### Red Flags\n- None triggered by the default heuristics.
 ↪"))

display(Markdown("### Saved outputs"))
saved_files = sorted(str(path.relative_to(OUTPUT_DIR)) for path in OUTPUT_DIR.
 ↪glob("*"))
display(pd.DataFrame({"file": saved_files}))
```

1.15.1 Report Summary

- Test: accuracy=0.994, precision=0.905, recall=0.856, f1=0.880.
- Unseen test: accuracy=0.987, precision=0.929, recall=0.845, f1=0.885.
- Unseen F1 change vs test is +0.005 (small gap). Unseen precision change is +0.024; unseen recall change is -0.011. Generalization to unseen CWE looks stable at the aggregate level.
- Weak CWEs by F1: CWE-399, CWE-476, CWE-125.
- Weak projects by F1: u-boot, poppler, gpac.
- Validation-based threshold tuning selected 0.70. Compare the tuned threshold against the default setting before changing production evaluation defaults.

1.15.2 Red Flags

- char_length bucket >4096 has high FN rate (0.55).

1.15.3 Saved outputs

	file
0	aggregate_metrics.csv
1	bootstrap_summary.csv
2	char_length_stratified_metrics.csv
3	confusion_matrix_test.png
4	confusion_matrix_unseen_test.png
5	confusion_matrix_unseen_val.png
6	confusion_matrix_val.png
7	line_count_stratified_metrics.csv
8	per_cwe_metrics.csv
9	per_cwe_metrics_test.csv
10	per_cwe_metrics_unseen_test.csv
11	per_cwe_metrics_unseen_val.csv
12	per_cwe_metrics_val.csv
13	per_cwe_support_vs_f1.png
14	per_cwe_support_vs_f1_test.png
15	per_cwe_support_vs_f1_unseen_test.png
16	per_cwe_support_vs_f1_unseen_val.png
17	per_cwe_support_vs_f1_val.png
18	per_cwe_support_vs_recall.png
19	per_project_metrics.csv
20	per_project_metrics_test.csv

```
             per_project_metrics_unseen_test.csv
              per_project_metrics_unseen_val.csv
                     per_project_metrics_val.csv
                   per_project_support_vs_f1.png
             per_project_support_vs_f1_test.png
      per_project_support_vs_f1_unseen_test.png
       per_project_support_vs_f1_unseen_val.png
              per_project_support_vs_f1_val.png
                           reliability_plot.png
                          reliability_table.csv
                              report_summary.md
                  score_distribution_labels.png
                score_distribution_outcomes.png
score_distribution_seen_vs_unseen_positives.png
                         seen_unseen_metrics.csv
                     seen_vs_unseen_metrics.png
                       test_vs_unseen_delta.csv
                       threshold_comparison.csv
                     threshold_sensitivity.png
                           threshold_sweep.csv
            token_length_stratified_metrics.csv
                         unseen_gap_aggregate.csv
        unseen_gap_positive_rate_by_project.csv
```